

Artificial Intelligence

CSC 309

Lecture Notes

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

Chapter 1: Introduction to Artificial Intelligence	3
Chapter 2: Intelligent Agents	12
Chapter 3: Search Algorithms and Problem Solving	26
Chapter 4: Knowledge Representation and Reasoning	34
Chapter 5: Natural Language Processing	45
Chapter 6: Expert Systems and AI Programming Languages	55
Chapter 7: Computer Vision and Robotics	66

Chapter 1

Introduction to Artificial Intelligence

1.1 Introduction to Artificial Intelligence

Artificial Intelligence (AI) represents one of the most transformative disciplines in modern computer science. It focuses on constructing computational systems capable of performing tasks that traditionally require human intelligence, such as logical reasoning, decision-making, visual perception, and natural language understanding. This chapter provides a comprehensive foundation for CSC 309 by exploring the historical evolution, core principles, operational methodologies, and major theoretical debates that define artificial intelligence.

Understanding the conceptual and historical origins of AI is essential for contextualizing modern computational approaches. By examining how symbolic reasoning gave way to probabilistic modeling and data-driven learning, students gain insight into why modern AI architectures are designed in specific ways. Furthermore, evaluating theoretical frameworks such as the Turing test and John Searle's Chinese Room thought experiment equips students with the critical analysis skills needed to evaluate system capabilities, performance boundaries, and ethical implications.

This introductory chapter serves as the foundation for the entire CSC 309 curriculum. The concepts, techniques, and taxonomy established here directly connect to subsequent chapters on intelligent agents, state-space search algorithms, formal knowledge representation, Natural Language Processing, expert systems, and robotics.

1.1.1 Definition and Primary Goals of Artificial Intelligence

Artificial Intelligence is defined as a subfield of computer science dedicated to creating hardware and software systems capable of performing tasks that typically demand human cognitive abilities, including learning, reasoning, problem-solving, perception, and language comprehension.

The field of AI is driven by two primary goals:

1. **Engineering Goal:** To solve complex, real-world problems by designing, building, and

deploying reliable, autonomous, and intelligent algorithms.

2. **Scientific Goal:** To model, formalize, and understand the mechanisms underlying cognitive processes and human intelligence through computational simulation.

To achieve these goals, artificial intelligence encompasses several specific sub-goals:

- Logical reasoning and automated decision-making under uncertain or partial information.
- Autonomous knowledge acquisition and continuous performance improvement through learning algorithms.
- Perceptual capabilities involving visual, auditory, and sensor-based environmental processing.
- Natural language interaction and semantic comprehension for seamless communication.

1.1.2 Historical Evolution of Artificial Intelligence

The historical trajectory of artificial intelligence spans several distinct eras, characterized by shifting paradigms, periods of high optimism, and subsequent periods of reduced funding known as AI winters.

[1940s–1950]	Foundational Roots (Turing’s Imitation Game)
[1956]	Dartmouth Workshop (Birth of Artificial Intelligence)
[1956–1970s]	Early Enthusiasm & Symbolic AI (GPS, ELIZA)
[1974–1980]	First AI Winter (Lighthill Report, combinatorial explosion)
[1980–1987]	Expert Systems Boom (Knowledge-based rule engines)
[1987–1993]	Second AI Winter (Hardware collapse, maintenance costs)
[1990s–Pres]	Modern Machine Learning & Deep Learning (Data-driven AI)

1. **Foundational Roots (1940s–1950):** Alan Turing laid the theoretical groundwork for machine intelligence. In his seminal 1950 paper, "Computing Machinery and Intelligence," Turing posed the fundamental question, "Can machines think?" and introduced the operational framework known as the Imitation Game.
2. **The Dartmouth Workshop (1956):** The formal birth of AI as an academic discipline occurred at the Dartmouth Summer Research Project on Artificial Intelligence organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. John McCarthy coined the term **Artificial Intelligence** during this historic gathering.
3. **Early Enthusiasm and Symbolic AI (1956–1970s):** Early AI research focused on symbolic logic, formal theorem proving, and micro-world problem solving. Systems such as the General Problem Solver (GPS) and ELIZA demonstrated early potential, leading to high optimism.

4. **The First AI Winter (1974–1980):** Early systems struggled with real-world complexity due to computational hardware constraints and combinatorial explosion. Critical reports, such as the Lighthill Report in the United Kingdom, led to widespread reductions in research funding.
5. **The Expert Systems Boom (1980–1987):** Industrial interest re-emerged with the adoption of knowledge-based systems. **Expert systems** used structured rules to emulate human domain expertise in specialized areas such as medical diagnostics and geological exploration.
6. **The Second AI Winter (1987–1993):** Specialized hardware markets collapsed, and rule-based expert systems proved expensive to maintain and difficult to scale to unmodeled scenarios.
7. **The Modern Era of Machine Learning and Deep Learning (1990s–Present):** Exponential increases in processing power, high-throughput memory architectures, and large datasets propelled a shift from hand-coded logical rules to statistical data-driven learning. Deep neural network architectures enabled breakthroughs across computer vision, speech recognition, and Natural Language Processing.

1.2 Fundamental AI Techniques and Taxonomy

1.2.1 Core AI Techniques

AI applications rely on four fundamental technical paradigms:

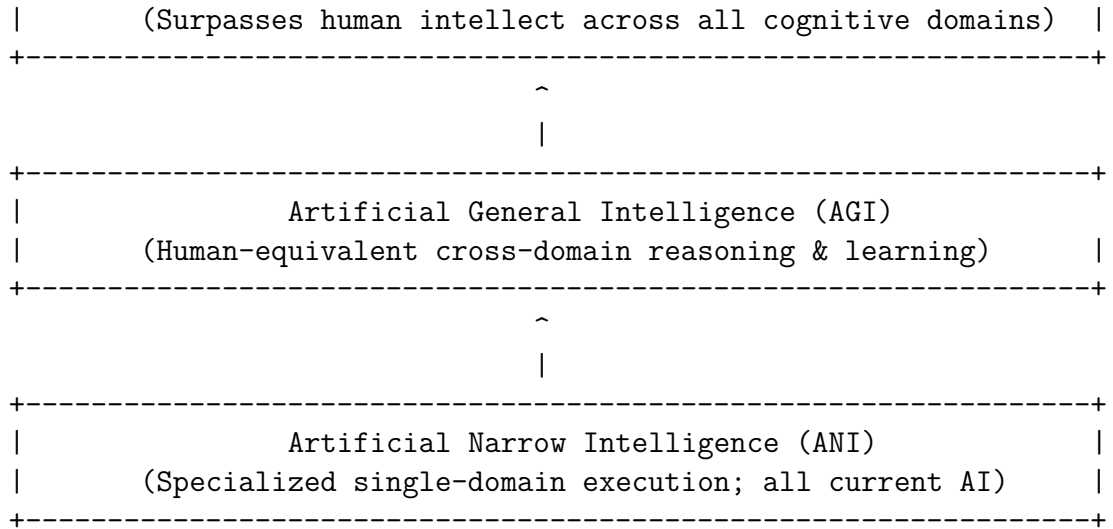
1. **Knowledge Representation and Logic:** Structuring domain facts, rules, and relationships into formal computational structures that permit automated inference and deduction.
2. **Search and Optimization:** Navigating vast state spaces systematically to identify optimal or satisfying action sequences using search algorithms and explicit evaluation metrics.
3. **Machine Learning and Statistical Pattern Recognition:** Applying mathematical models that automatically infer patterns, underlying distributions, and decision boundaries from training data without explicit manual programming.
4. **Neural Networks and Deep Learning:** Processing complex high-dimensional data through layered computational graphs that extract hierarchical representations directly from raw inputs.

1.2.2 Classification and Types of Artificial Intelligence

Artificial intelligence systems are classified according to their capability levels and functional architectures.

Classification by Capability Level

+-----+
 | Artificial Superintelligence (ASI) |



- **Artificial Narrow Intelligence (ANI):** Also referred to as Weak AI, ANI describes systems built and optimized to perform a specific, isolated task (e.g., chess software, spam filters, or image recognition models). ANI systems cannot transfer their functional expertise to unassigned tasks. All operational AI systems today belong to this category.
- **Artificial General Intelligence (AGI):** Also known as Strong AI, AGI refers to theoretical systems possessing human-level cognitive flexibility. An AGI system could perform cross-domain reasoning, transfer learning across disparate subjects, and adapt dynamically to unfamiliar environments.
- **Artificial Superintelligence (ASI):** A theoretical extension of AGI where machine cognitive abilities surpass human intellect across all computational, creative, scientific, and social domains.

Classification by Functional Architecture

- **Reactive Machines:** Basic systems that calculate actions purely from current environmental inputs without maintaining internal memory or past state history (e.g., IBM's Deep Blue).
- **Limited Memory:** Systems capable of retaining dynamic short-term memory to inform real-time decision-making (e.g., trajectory planning in autonomous vehicle navigation).
- **Theory of Mind:** Theoretical frameworks capable of representing and reasoning about the psychological states, intentions, beliefs, and emotions of human agents.
- **Self-Aware AI:** Theoretical systems possessing explicit self-consciousness, self-identity, and self-referential mental monitoring.

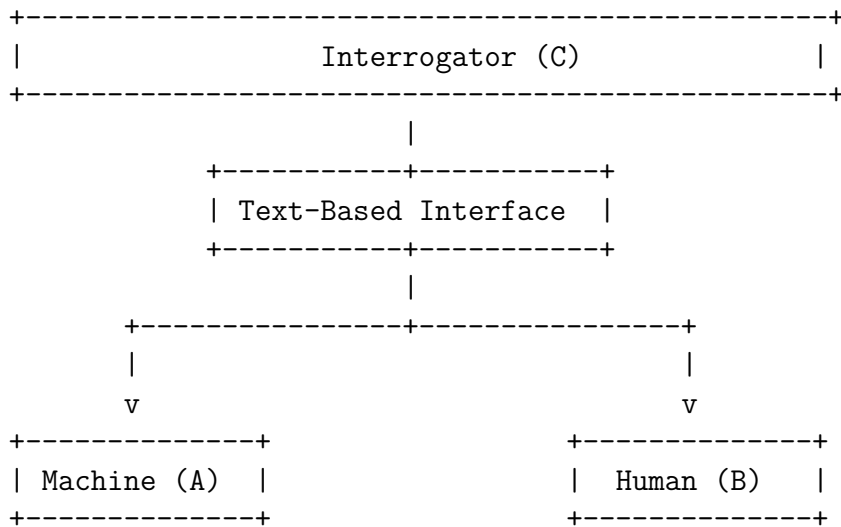
1.3 Evaluating Machine Intelligence

1.3.1 The Turing Test

The **Turing test**, proposed by Alan Turing in 1950, provides an operational methodology for evaluating whether a system can display behavior indistinguishable from human intelligence.

Category	Key Characteristics	Current Operational Status
Artificial Narrow Intelligence (ANI)	Programmed for specialized, single-domain execution; incapable of cross-domain transfer.	Deployed universally in production (e.g., search engines, diagnostics).
Artificial General Intelligence (AGI)	Human-equivalent cognitive versatility; adapts autonomously across open domains.	Theoretical research objective; no physical instances exist.
Artificial Super-intelligence (ASI)	Cognitive capacity exceeding human capability across all problem domains.	Hypothetical concept; focus of safety and ethics research.

In the standard operational setup, a human interrogator (C) communicates with two hidden entities through a text-based terminal interface: a human (B) and a machine (A). The interrogator submits text queries and evaluates the text responses. The machine's goal is to deceive the interrogator into believing it is the human. If the interrogator cannot reliably distinguish the machine from the human after a specified evaluation duration, the machine passes the test.



Despite its historical significance, the Turing test faces critical theoretical objections:

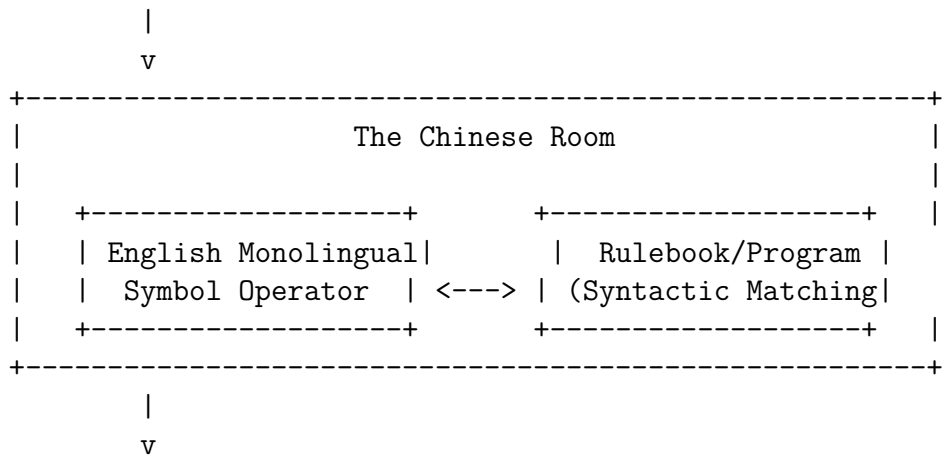
- It measures external behavioral imitation rather than underlying mental cognition or comprehension.
- Simple trickery, pre-programmed evasion techniques, or superficial surface matching can deceive human interrogators.
- It is anthropocentric, evaluating intelligence strictly through the lens of human linguistic behavior.

1.3.2 The Chinese Room Thought Experiment

Formulated by philosopher John Searle in 1980, the **Chinese Room thought experiment** challenges the claim that functional symbol manipulation constitutes true cognitive understanding or intentionality.

Searle asks us to imagine a monolingual English speaker who knows no Chinese, locked inside a sealed room. The room contains a structured rulebook written in English. Outside observers slide slips of paper containing Chinese symbols into the room through a slot. The occupant uses the English rulebook to look up incoming Chinese character sequences and follow exact instructions for matching them to corresponding output Chinese characters. The occupant then slides the output slips back out through a second slot.

Input: Chinese Symbols



Output: Chinese Symbols

To an outside Chinese speaker, the responses emerging from the room are coherent and intelligent. However, the person inside the room processes the symbols purely based on syntactic rules without understanding any semantic meaning of the Chinese characters.

Searle uses this experiment to make a key distinction:

- **Syntax:** Formal rules governing symbol manipulation and computational structure.
- **Semantics:** The actual meaning, intentionality, and underlying understanding associated with symbols.

Searle concludes that digital computers operate entirely syntactically. Consequently, running a program, regardless of its operational complexity, cannot produce genuine semantic understanding, mental states, or intentionality.

1.4 Branches, Applications, and Operational Trade-Offs

1.4.1 Key Branches and Real-World Applications

Artificial intelligence spans multiple primary subfields:

Dimension	Turing Test	Chinese Room Thought Experiment
Primary Proponent	Alan Turing (1950)	John Searle (1980)
Core Criterion	External operational output and conversational capability.	Internal semantic state and conscious intentionality.
Main Conclusion	Behavior indistinguishable from human intelligence implies cognitive capability.	Correct symbol manipulation can occur entirely without semantic understanding.
Philosophical View	Functionalism / Behaviorism.	Biological Naturalism / Anti-functionalism.

- **Natural Language Processing (NLP):** Algorithms designed for processing, understanding, translating, and generating natural human language text and speech.
- **Computer Vision:** Techniques for capturing, transforming, and interpreting visual information from digital image streams or camera arrays.
- **Robotics:** Integration of perceptual sensors, decision-making controllers, and physical actuators to carry out tasks in dynamic real-world environments.
- **Expert Systems:** Knowledge-based software engines that use rule structures to simulate domain-specific human decision-making.
- **Machine Learning:** Statistical modeling techniques that enable systems to improve task performance automatically through data experience.

AI technologies are deployed across major industrial domains:

- **Healthcare:** Automated diagnostic image interpretation, personalized genomic analysis, and predictive clinical risk modeling.
- **Finance:** High-frequency algorithmic stock trading, real-time banking transaction fraud detection, and credit assessment models.
- **Transportation:** Autonomous vehicle obstacle avoidance, dynamic urban traffic lights regulation, and supply chain path optimization.
- **E-Commerce:** Dynamic user recommendation engines, inventory management, and automated customer support interfaces.

1.4.2 Advantages and Disadvantages of Artificial Intelligence

Deploying artificial intelligence systems involves significant operational advantages alongside structural risks.

1.5 Conclusion

This chapter established the foundational history, core techniques, functional taxonomies, and theoretical questions surrounding artificial intelligence. While empirical metrics such as the

Advantages	Disadvantages and Risks
Continuous 24/7 operational availability without fatigue or performance degradation.	High initial capital expenditure, ongoing infrastructure costs, and energy consumption.
High-speed data evaluation and statistical accuracy across massive datasets.	System susceptibility to algorithmic bias inherited from skewed training data.
Automation of hazardous, repetitive, or error-prone tasks.	Structural workforce dislocation and job displacement across various sectors.
Consistent execution of complex rules without human variance.	Complete absence of moral judgment, human empathy, and genuine common-sense context.

Turing test and arguments like the Chinese Room thought experiment explore the boundary between behavioral simulation and genuine cognitive understanding, practical AI systems rely on targeted techniques, such as search algorithms, knowledge representation, and statistical machine learning, to solve real-world problems.

In Chapter 2, we formalize how these individual AI capabilities are unified into structured systems known as **intelligent agents**, analyzing how agents interact with diverse operational environments to maximize performance metrics.

Tutorial Questions

1. Distinguish between the engineering goal and the scientific goal of artificial intelligence. Provide a practical application scenario illustrating each goal.
2. Outline the primary factors that caused the first AI winter during the 1970s. How did the transition from symbolic AI to data-driven machine learning address these systemic limitations?
3. Compare Artificial Narrow Intelligence (ANI), Artificial General Intelligence (AGI), and Artificial Superintelligence (ASI). Why are all current operational AI systems classified as ANI?
4. Describe the operational configuration of the Turing test. Discuss two major conceptual criticisms of using the Turing test as a definitive metric for genuine intelligence.
5. Explain John Searle's Chinese Room thought experiment. How does Searle use this experiment to differentiate between syntax and semantics, and what are its implications for Strong AI?
6. Compare reactive machines with limited memory AI systems, giving one concrete example of each system type.
7. An automated diagnostic system is trained on historical hospital records to flag high-risk patient admissions. Identify two operational advantages and two technical or ethical risks associated with deploying this AI system.
8. Explain why rule-based expert systems struggled to scale when applied to complex real-world environments, and describe how statistical machine learning techniques overcome these scalability barriers.

Chapter 2

Intelligent Agents

2.1 Overview of Intelligent Agents

This chapter introduces the concept of intelligent agents, which serve as the foundational unifying framework for modern artificial intelligence. It examines how agents perceive their surroundings through sensors, process information, and execute actions through actuators within diverse operational environments. The chapter explores the criteria for agent rationality, formalizes task environment specifications using the Performance, Environment, Actuators, and Sensors (PEAS) framework, and analyzes fundamental agent architectures ranging from simple reflex systems to sophisticated learning agents.

Understanding intelligent agents is critical because it shifts the focus of artificial intelligence from isolated algorithms to holistic, goal-oriented decision-making systems. Rather than viewing intelligence merely as a collection of specialized techniques, such as search or logic, the agent-centric viewpoint provides a principled structure for designing autonomous systems that operate effectively in complex, dynamic, and uncertain real-world domains. Studying agent architectures allows engineers and computer scientists to evaluate trade-offs between computational overhead, reactivity, adaptability, and decision accuracy.

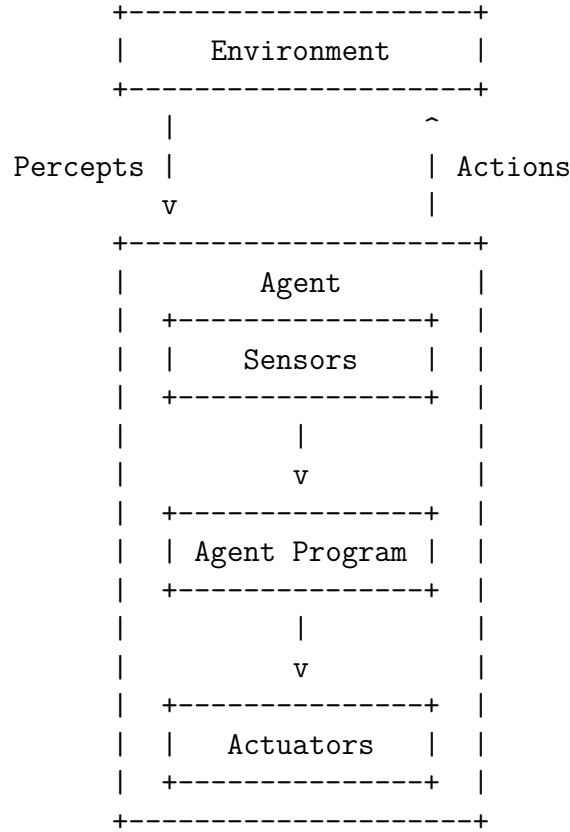
Building upon the historical and theoretical foundations established in Chapter 1, this chapter establishes the design paradigm used throughout the remainder of CSC 309. The environment classifications and structural components introduced here provide the context for problem-solving by search algorithms in Chapter 3, formal Knowledge representation models in Chapter 4, and specialized domain applications such as Natural Language Processing, expert systems, and computer vision in subsequent chapters.

2.1.1 Definition and Core Concepts

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators. A human agent possesses physical sensors such as eyes, ears, and nose, alongside actuators such as hands, legs, and vocal cords. A robotic agent replaces biological sensors with cameras, infrared rangefinders, and lidar units, and replaces biological actuators with electric motors, hydraulic limbs, and grippers.

A software agent receives inputs in the form of file contents, network packets, or database queries, and acts upon its environment by writing files, sending network packets, or updating database records.

An **intelligent agent** is an agent that selects actions expected to maximize its performance measure based on its percept sequence and built-in knowledge.



To formalize agent operation, four underlying concepts are defined:

- **Percept:** The agent's perceptual inputs at any given instant.
- **Percept Sequence:** The complete history of everything the agent has ever perceived during its lifetime.
- **Sensors:** The physical or virtual devices through which the agent receives percepts from the environment.
- **Actuators:** The mechanisms through which the agent executes actions to modify the state of the environment.

2.1.2 Agent Functions and Agent Programs

Mathematically, an agent's behavior is described by the agent function, which maps any given percept sequence to a corresponding action. Let $\mathcal{P}$ denote the set of all possible single percepts, and let $\mathcal{P}^*$ represent the set of all possible percept sequences. Let $\mathcal{A}$ denote the set

of all possible actions. The agent function f is defined as:

$$f : \mathcal{P}^* \rightarrow \mathcal{A} \quad (2.1)$$

The agent function is an abstract mathematical mapping. In practice, the agent function is implemented by an agent program, which runs on a physical computing infrastructure known as the **agent architecture**.

While the agent function takes the entire history of percepts $\mathcal{P}^*$ as input, the agent program accepts only the current percept $p \in \mathcal{P}$ as an input from the sensors because storing and searching an unbounded percept sequence directly is computationally infeasible. The agent program maintains internal memory structures to represent relevant aspects of past percepts when necessary.

To illustrate this distinction, consider a simplified automated vacuum cleaner domain containing two locations, designated as Room A and Room B. The agent perceives its current location and whether that location contains dirt. The percept is represented as a pair, such as (Location A, Clean) or (Location B, Dirty). The available actions are Move Left, Move Right, Suck, and No-Op.

Percept Sequence	Action Choice
[(Location A, Clean)]	Move Right
[(Location A, Dirty)]	Suck
[(Location B, Clean)]	Move Left
[(Location B, Dirty)]	Suck
[(Location A, Clean), (Location B, Dirty)]	Suck
[(Location A, Clean), (Location B, Clean)]	No-Op

Table 2.1: Partial tabular representation of an agent function for a two-room vacuum world.

Table 2.1 demonstrates that the agent function grows exponentially with time if represented as a lookup table. For an agent operating over t time steps with $|\mathcal{P}|$ possible percepts, the number of entries in a complete lookup table is given by:

$$\sum_{k=1}^t |\mathcal{P}|^k = \frac{|\mathcal{P}|(|\mathcal{P}|^t - 1)}{|\mathcal{P}| - 1} \quad (2.2)$$

This exponential growth illustrates why explicit lookup tables are intractable for real-world artificial intelligence applications, necessitating structured agent programs.

2.2 Rationality and Agent Performance

Rationality evaluates whether an agent makes appropriate decisions given what it perceives and what it knows. Rationality is distinct from perfection.

2.2.1 Performance Measures

A performance measure is an objective criterion used to evaluate how successful an agent's behavior is within a given environment. It is crucial that performance measures are defined in terms of desired outcomes in the environment rather than in terms of the agent's internal state or actions.

For example, evaluating a vacuum cleaner agent based on the amount of dirt it collects might encourage an agent to repeatedly dump dirt and suck it back up to maximize its score. A sound performance measure evaluates the clean state of the environment over time, penalizing excessive energy consumption and noise.

2.2.2 Defining Rationality

What is rational at any given time depends on four distinct factors:

1. The performance measure that defines the criterion of success.
2. The agent's prior knowledge of the environment.
3. The actions that the agent can perform.
4. The agent's percept sequence to date.

Based on these factors, a rational agent is defined formally as follows: For each possible percept sequence, a rational agent selects an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent possesses.

Mathematically, if S represents the set of possible environment states, $U(s)$ denotes the utility or performance score of state $s \in S$, and $P(s \mid \mathcal{P}^*, a)$ represents the probability distribution over resulting states given percept sequence $\mathcal{P}^*$ and selected action $a \in \mathcal{A}$, the rational action choice a^* satisfies:

$$a^* = \arg \max_{a \in \mathcal{A}} \sum_{s \in S} P(s \mid \mathcal{P}^*, a) U(s) \quad (2.3)$$

2.2.3 Rationality versus Omniscience

Rationality is not synonymous with omniscience. An omniscient agent knows the actual outcome of its actions and acts accordingly. Omniscience is impossible in real-world applications because environments are uncertain and incomplete information is inherent to perceptual mechanisms.

Rationality maximizes **expected** performance, whereas omniscience maximizes **actual** performance. An agent decision that leads to an undesirable outcome due to unforeseen events is still rational if the chosen action was optimal based on the evidence available prior to execution.

Consider an individual walking down a designated pedestrian pathway who is struck by a falling piece of aircraft debris. The decision to walk along the path remains rational

because prior knowledge and sensory inputs provided no reasonable basis to predict the event. Expecting the agent to have avoided the location demands omniscience, not rationality.

2.2.4 Autonomy and Learning

An agent demonstrates autonomy to the extent that its choices depend on its own experience rather than on pre-programmed designer knowledge. An agent that relies entirely on built-in assumptions lacks flexibility and fails when deployed in changing environments.

A rational agent must possess two secondary mechanisms to support long-term operational success:

- **Information Gathering:** Taking actions specifically designed to expand the percept sequence and reduce uncertainty, such as looking around before crossing a street.
- **Learning:** Modifying initial knowledge structures based on environmental feedback over time, transforming a non-autonomous agent into an autonomous system.

2.3 Task Environments and the PEAS Framework

In agent design, the problem specification is termed the task environment. The framework used to specify a task environment is known as the PEAS framework.

2.3.1 The PEAS Framework

PEAS is an acronym representing four critical components:

- **Performance Measure:** The explicit metrics used to rate agent success.
- **Environment:** The external medium, domain, or context within which the agent operates.
- **Actuators:** The set of tools or output interfaces through which the agent acts.
- **Sensors:** The set of devices or input interfaces through which the agent receives percepts.

To design an effective agent program, the PEAS specification must first be clearly articulated.

2.3.2 Environment Properties

Task environments exhibit key structural dimensions that dictate the complexity of the required agent architecture:

Fully Observable versus Partially Observable

An environment is fully observable if an agent's sensors provide access to the complete state of the environment at each point in time. An environment is partially observable if sensors are noisy, inaccurate, or unable to detect aspects of the state due to physical limitations or

Agent Type	Performance	Environment	Actuators	Sensors
Automated Taxi Driver	Safety, speed, legal compliance, passenger comfort, profit	Urban roads, vehicular traffic, pedestrians, weather conditions	Steering, accelerator, brakes, signals, horn, display screen	Cameras, radar, lidar, GPS, accelerometer, engine sensors
Medical Diagnosis System	Correct diagnosis, minimized treatment costs, patient health	Patient body, clinical laboratory, healthcare staff	Displayed diagnosis, test recommendations, treatment plans	Keyboard entry of symptoms, laboratory test results
Satellite Image Classifier	Classification accuracy, throughput, low processing latency	Continuous satellite imagery stream	Categorization labels, anomaly warning flags	High-resolution optical sensors, infrared channels
Interactive English Tutor	Maximized student test performance, user engagement	Student cohort, curriculum database	Screen display of exercises, feedback text, speech output	Keyboard input, touchscreen interactions, microphone

Table 2.2: PEAS specifications across diverse application domains.

occlusions. In a partially observable domain, the agent must maintain an internal state to track unobserved elements.

Single-Agent versus Multi-Agent

An environment is single-agent if an entity acts alone without other decision-making agents affecting performance. An environment is multi-agent if other agents exist whose actions affect the performance measure of the primary agent. Multi-agent environments are categorized as competitive (such as chess) or cooperative (such as collaborative traffic management systems).

Deterministic versus Stochastic

An environment is deterministic if the next state of the environment is completely determined by the current state and the action executed by the agent. If uncertainty exists regarding the resulting state due to random elements or unobserved dynamics, the environment is stochastic. If the environment is not deterministic but outcomes are listed without explicitly attached probabilities, it is termed nondeterministic.

Episodic versus Sequential

In an episodic environment, the agent’s experience is divided into atomic episodes. Each episode consists of the agent perceiving and performing a single action. Crucially, the action taken in one episode does not affect decisions made in subsequent episodes. In a sequential environment, current decisions influence all future decisions, requiring long-term planning.

Static versus Dynamic

An environment is static if it does not change while the agent is deliberating. An environment is dynamic if it changes continuously while the agent processes information. If the environment itself does not change with time, but the agent’s performance score deteriorates as deliberation time passes, the environment is classified as semidynamic.

Discrete versus Continuous

An environment is discrete if it possesses a finite or countably infinite number of distinct states, percepts, and actions. An environment is continuous if states, percepts, or actions vary continuously over real numbers, such as vehicle positioning, velocity, and steering angles.

Known versus Unknown

This dimension refers to the agent designer’s state of knowledge regarding the environment’s operational laws. In a known environment, the outcomes or outcome probabilities for all actions are mathematically specified. In an unknown environment, the agent must learn how the environment functions through direct trial and interaction.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Chess (with-out clock)	Fully	Multi	Deterministic	Sequential	Static	Discrete
Chess (with clock)	Fully	Multi	Deterministic	Sequential	Semidynamic	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Automated Taxi Driver	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical Diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Part-Sorting Robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Image Analysis	Fully	Single	Deterministic	Episodic	Semidynamic	Discrete

Table 2.3: Classification of sample task environments across major structural dimensions.

2.4 Agent Architectures and Program Design

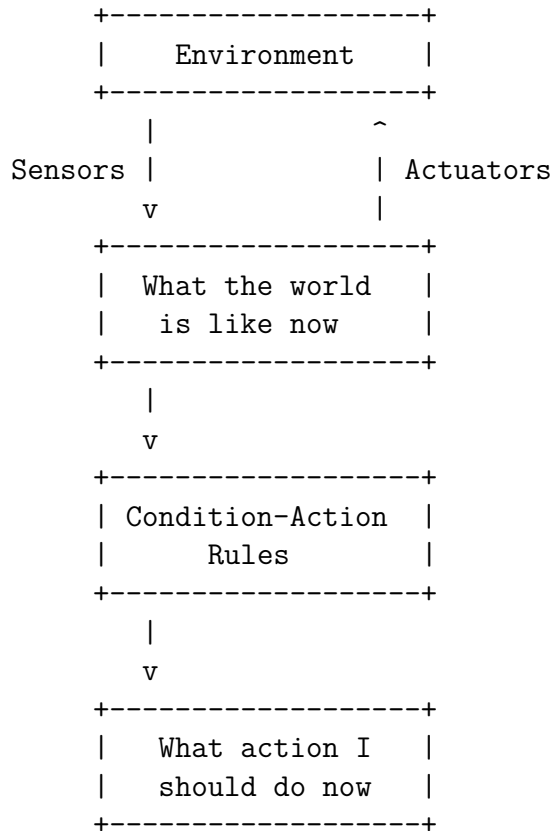
An agent architecture provides the physical computing system and sensor-actuator interface that executes the agent program. The relation between these components is expressed as:

$$\text{Agent} = \text{Architecture} + \text{Program} \quad (2.4)$$

The underlying structure of the agent program determines how percepts are transformed into concrete action sequences. Five basic agent architecture structures exist in artificial intelligence.

2.4.1 Simple Reflex Agents

Simple reflex agents select actions based exclusively on the current percept, ignoring the entire percept history. Their decision logic is governed by condition-action rules, often framed as simple if – then constructs.



The simple reflex agent function operates as follows:

```

function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition-action rules

  state <- INTERPRET-INPUT(percept)
  rule <- RULE-MATCH(state, rules)
  action <- rule.ACTION
  return action

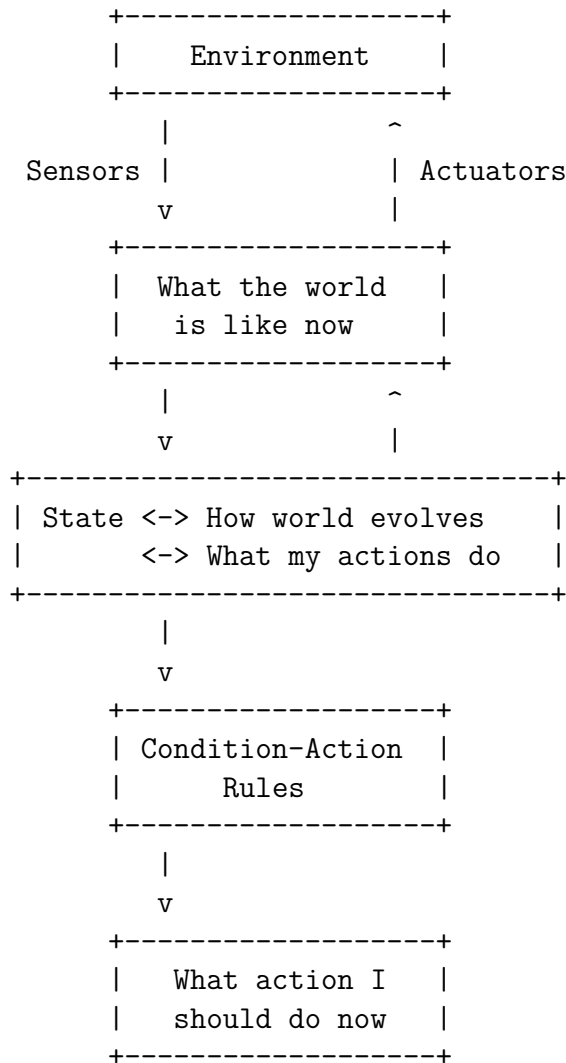
```

Simple reflex agents are limited because they can only operate successfully in environments that are fully observable. If deployed in partially observable domains, simple reflex agents frequently enter infinite loops.

2.4.2 Model-Based Reflex Agents

To handle partial observability, a model-based reflex agent maintains an internal state that tracks unobserved aspects of the current world. To maintain this state, the agent requires two types of knowledge encoded within its model:

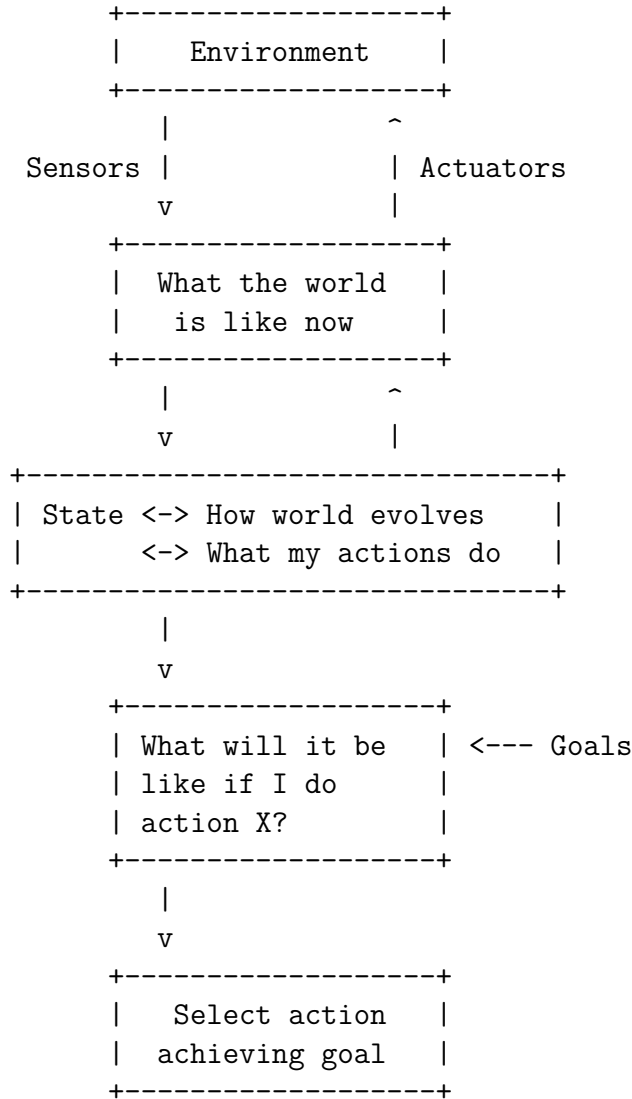
1. **Transition Model:** Knowledge about how the world evolves independently of the agent, and how the agent's actions affect the world.
2. **Sensor Model:** Knowledge about how the state of the world maps to sensory percepts.



The model-based agent updates its internal state representation over time using incoming percepts and its transition knowledge before selecting an action.

2.4.3 Goal-Based Agents

Knowing the current state of the environment is not always sufficient to decide what action to select. A goal-based agent combines internal state tracking with explicit goal specifications that describe desirable environment situations.

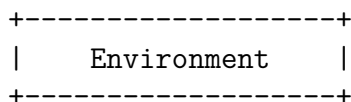


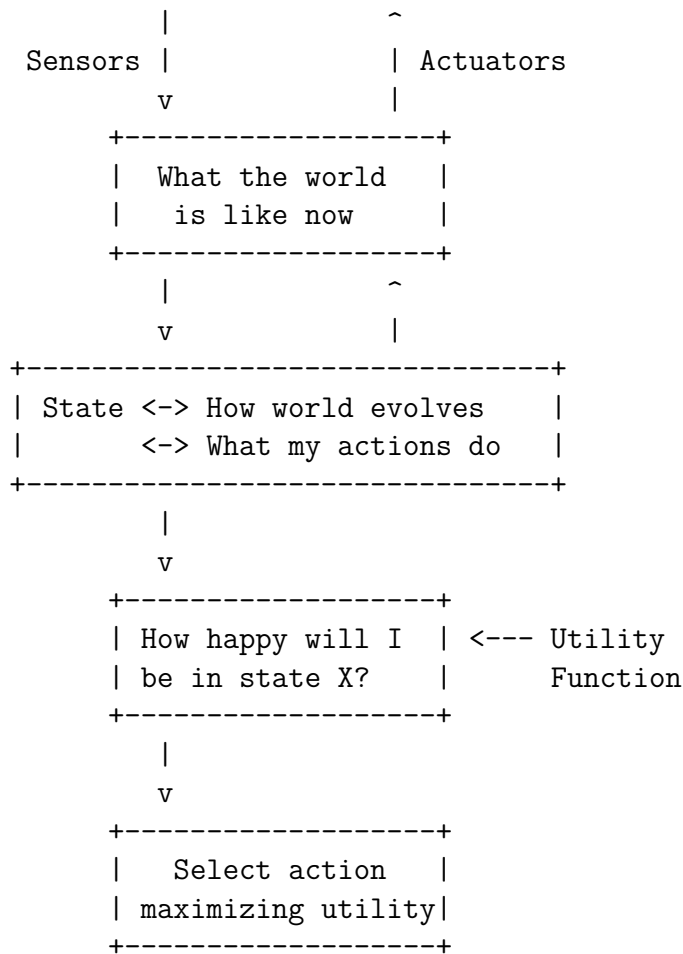
Goal-based architectures require deliberative mechanisms such as search algorithms and planning sequences. Unlike reflex agents, goal-based agents evaluate future outcomes before selecting an action, making them far more flexible when conditions change.

2.4.4 Utility-Based Agents

Goals provide a binary distinction between goal states and non-goal states. However, real-world decision-making requires evaluating trade-offs between competing outcomes, such as balance between speed, safety, and fuel efficiency in automated driving.

A utility-based agent utilizes a utility function that maps an environment state (or sequence of states) to a real number $U : S \rightarrow \mathbb{R}$. This numerical score measures how desirable a given state is for the agent.



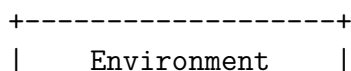


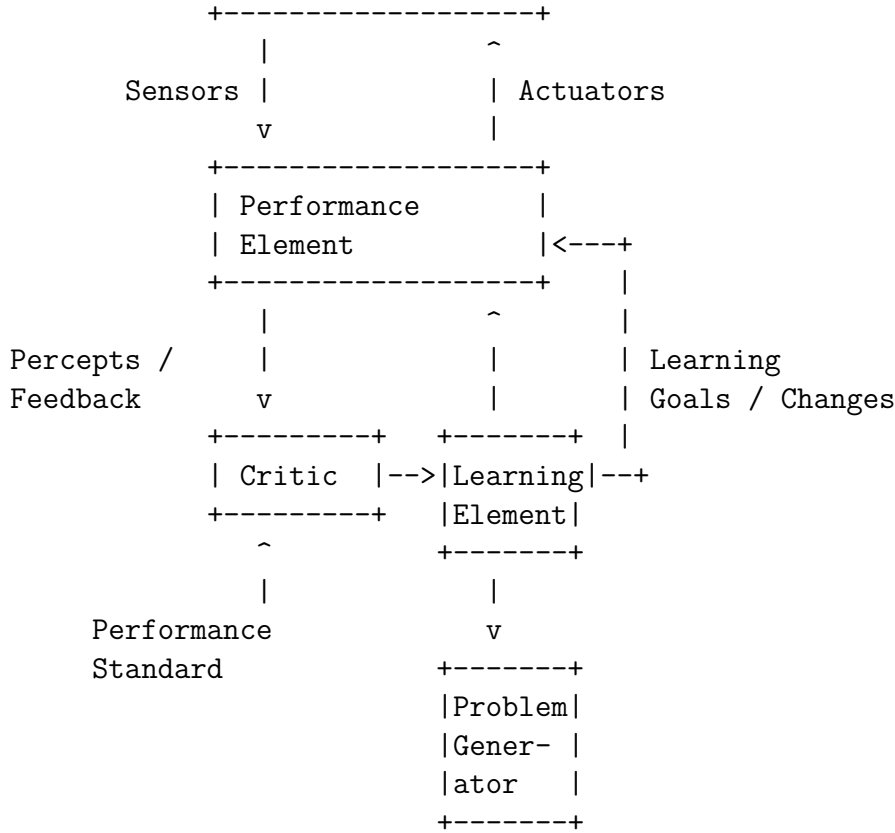
A utility-based agent selects actions that maximize expected utility, providing a principled basis for decision-making under uncertainty and resolving conflicts between mutually exclusive objectives.

2.4.5 Learning Agents

Learning agents adapt their internal decision logic automatically over time. A learning agent architecture is conceptually divided into four separate operational components:

- **Learning Element:** Responsible for making structural improvements to the agent program based on experience.
- **Critic:** Evaluates the agent's behavior against an external performance standard and provides evaluative feedback to the learning element.
- **Performance Element:** The component responsible for selecting external actions based on percepts; equivalent to the entire agent programs described in previous architectures.
- **Problem Generator:** Suggests exploratory actions that lead to new experiences rather than suboptimal actions, prioritizing discovery over short-term rewards.





2.4.6 Comparative Analysis of Agent Architectures

Table 2.4 summarizes the structural trade-offs between the five fundamental agent architectures.

Architecture	State Representation	Decision Mechanism	Best Suited Environment
Simple Reflex	None (Current percept only)	Condition-action rule lookup	Fully observable, static, simple domains
Model-Based Reflex	Internal state history	Rules evaluated over world model state	Partially observable domains with clear state logic
Goal-Based	Internal state history	Search and planning toward explicit goals	Complex domains requiring multi-step lookahead
Utility-Based	Internal state history	Utility optimization under probabilistic outcomes	Dynamic domains with conflicting trade-offs
Learning Agent	Dynamic adaptivity across components	Learning element guided by critic and problem generator	Unknown, non-stationary, complex environments

Table 2.4: Comparative breakdown of fundamental agent architectures.

2.5 Conclusion

The conceptual paradigm of intelligent agents provides a unified foundation for artificial intelligence. By formalizing agent interactions using the PEAS framework, defining rationality through expected performance maximization, and selecting appropriate agent architectures based on environmental dimensions, complex autonomous systems can be designed systematically. The agent framework established in this chapter serves as the theoretical foundation for Chapter 3, where specific algorithmic techniques for state space search and problem-solving are developed.

Tutorial Questions

1. Define an intelligent agent and explain the theoretical distinction between an agent function and an agent program. Why is implementing an agent function as a lookup table infeasible in practice?
2. Differentiate between rationality and omniscience using a concrete operational example. Identify the four factors that dictate rational behavior for an agent.
3. Specify the task environment for an automated medical diagnosis agent using the PEAS framework.
4. Classify the following task environments across all seven environmental dimensions:
 - (a) An automated warehouse forklift picking and moving boxes.
 - (b) An online fraud detection system monitoring credit card transactions.
 - (c) A software agent playing an online multi-player strategy game.
5. A simple reflex agent is deployed in a partially observable environment. Explain why this agent is vulnerable to getting stuck in infinite loops. Provide a small abstract environment scenario illustrating this failure mode.
6. Contrast goal-based agents with utility-based agents. Describe a specific scenario where a goal-based agent fails to yield an optimal decision, whereas a utility-based agent succeeds.
7. Detail the four main internal components of a learning agent architecture. Explain the role played by the problem generator component and why pure performance optimization can be detrimental to an agent's long-term adaptability.

Chapter 3

Search Algorithms and Problem Solving

An intelligent agent must choose actions that lead to desirable goal states. When an agent cannot directly perceive the immediate sequence of actions required to achieve its objective, it must formulate a plan through problem solving by search. Search algorithms systematically evaluate potential sequences of actions within a state space to discover a sequence that transitions the system from an initial state to a goal state.

Search algorithms serve as a foundational mechanism across artificial intelligence, underpinning automated reasoning, game playing, route planning, and optimization. Understanding search strategies enables system designers to evaluate trade-offs between computational resource consumption, solution accuracy, and execution speed.

This chapter examines the core principles of search-based problem formulation, classifies search strategies into uninformed and informed approaches, formalizes heuristic evaluation functions, and analyzes performance using time and space complexity. Finally, the chapter analyzes the phenomenon of combinatorial explosion and explores techniques used to manage state space growth.

3.1 Problem Solving by Search

Problem solving by search relies on an abstract representation of the environment. Before an agent can execute a search algorithm, it must formulate the problem by defining states, actions, transitions, and evaluation metrics.

3.1.1 Formulation of Search Problems

A search problem is formally defined by five structural components:

1. **Initial State:** The state in which the agent begins its operation.
2. **Actions:** The set of allowable operations available to the agent. Given a state s , $Actions(s)$ denotes the set of actions that can be executed in s .

3. **Transition Model:** A function $Result(s, a)$ that returns the state resulting from executing action a in state s .
4. **Goal Test:** A boolean test that determines whether a given state satisfies the goal criteria. This may be an explicit set of goal states or an implicit condition.
5. **Path Cost:** A numerical function $c(s, a, s')$ that assigns a positive cost to traversing from state s to state s' via action a . The total path cost $g(n)$ is the sum of step costs along the path from the initial state to node n .

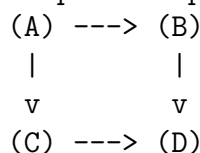
A solution to a search problem is an action sequence that leads from the initial state to a goal state. An optimal solution is a solution that achieves the lowest total path cost among all possible solutions.

3.1.2 State Space and Search Trees

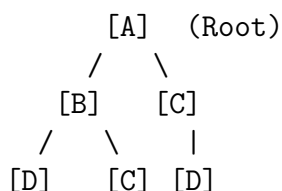
The **state space** is the set of all states reachable from the initial state through any sequence of valid actions. The state space forms a directed graph in which nodes represent physical or logical configurations of the system, and directed edges represent valid action transitions.

A **search tree** is an explicit data structure constructed during the search process to explore the state space. The root of the search tree corresponds to the initial state. Expanding a node involves applying all valid actions to its state, generating child nodes corresponding to successor states. A node in a search tree differs from a state in the state space: a search node contains additional bookkeeping information, including its parent node, the action applied, the path cost $g(n)$, and the search depth.

State Space Graph



Search Tree



3.2 Classes of Search Algorithms

Search algorithms are broadly categorized according to the information available to guide the exploration process. The two main categories are **uninformed search** (blind search) and **informed search** (heuristic search).

3.3 Uninformed Search Strategies

Uninformed search strategies explore the state space without domain-specific estimates of goal proximity. Nodes are expanded strictly based on the order in which they are generated or their distance from the start node.

Property	Uninformed Search	Informed Search
Domain Knowledge	Uses only problem formulation components (initial state, actions, transition model, goal test).	Uses domain-specific estimates (heuristics) to estimate distance to the goal.
Node Selection	Expands nodes strictly based on structural depth or path cost accumulated so far.	Expands nodes based on evaluation functions that combine accumulated cost and estimated remaining cost.
Efficiency	Explores the state space systematically without direction, leading to higher search time in large spaces.	Guides exploration toward promising areas of the state space, reducing search overhead.
Examples	Breadth-First Search, Depth-First Search, Uniform-Cost Search.	Greedy Best-First Search, A* Search.

Table 3.1: Comparison of Uninformed and Informed Search Classes

3.3.1 Breadth-First Search

Breadth-First Search (BFS) expands the shallowest unexpanded node in the search tree. BFS uses a First-In, First-Out (FIFO) queue to manage the frontier of unexpanded nodes.

Algorithm steps:

1. Initialize the frontier with a node containing the initial state.
2. If the frontier is empty, return failure.
3. Remove the shallowest node from the frontier.
4. If the node state satisfies the goal test, return the corresponding solution path.
5. Otherwise, expand the node and add all successor nodes not previously generated to the frontier.

BFS is complete when the branching factor b is finite. It guarantees finding an optimal solution if every step cost is identical.

3.3.2 Depth-First Search

Depth-First Search (DFS) expands the deepest unexpanded node in the current frontier. DFS uses a Last-In, First-Out (LIFO) queue (stack) for the frontier.

DFS traverses deep along a single path until a node has no successors or a goal state is hit, then backtracks to explore alternative paths. While DFS is not optimal and can fail to terminate in state spaces with infinite depth or cycles, its memory footprint is significantly smaller than BFS because it stores only the active path and unexplored sibling nodes.

3.3.3 Uniform-Cost Search

Uniform-Cost Search (UCS) generalizes BFS for state spaces with varying step costs. Instead of expanding shallowest nodes, UCS expands the node n with the lowest accumulated path cost $g(n)$. The frontier is implemented using a priority queue ordered by $g(n)$.

UCS applies the goal test when a node is selected for expansion, rather than when it is generated. This guarantees that when a goal node is expanded, no lower-cost path to that goal exists. UCS is optimal provided every step cost strictly exceeds a positive constant $\epsilon > 0$.

3.3.4 Depth-Limited and Iterative Deepening Search

Depth-Limited Search (DLS) mitigates the infinite-path failure of DFS by imposing a maximum search depth limit l . Nodes at depth l are treated as if they have no successors. If l is chosen smaller than the optimal solution depth d , DLS fails to find a solution.

Iterative Deepening Search (IDS) resolves the limit selection problem by running DLS repeatedly with increasing depth limits ($l = 0, 1, 2, \dots$) until a goal state is discovered. IDS combines the space efficiency of DFS with the completeness and optimality guarantees of BFS for uniform step costs. Although IDS re-generates top-level nodes across iterations, the computational overhead is minimal because the majority of nodes in a tree reside at the deepest level.

3.4 Heuristics and Informed Search Strategies

Informed search strategies incorporate domain knowledge using **heuristics** to prioritize node expansion.

3.4.1 Heuristic Functions

A heuristic function, denoted $h(n)$, estimates the cost of the cheapest path from the state at node n to a goal state:

$$h(n) = \text{estimated cost from node } n \text{ to a goal}$$

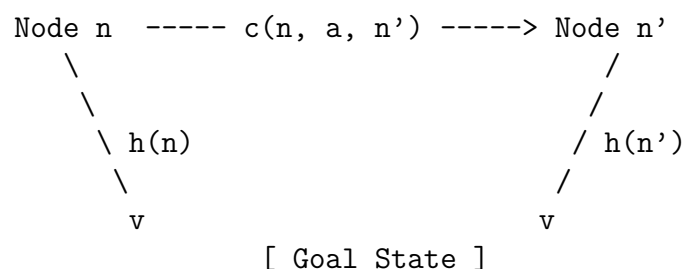
Heuristic functions are domain-dependent and satisfy $h(n) = 0$ at any goal state.

To guarantee that an informed search algorithm finds an optimal solution, heuristic functions must satisfy formal structural properties:

1. **Admissibility:** A heuristic function $h(n)$ is admissible if it never overestimates the actual minimal cost to reach a goal state from node n . That is, $0 \leq h(n) \leq h^*(n)$ for all nodes n , where $h^*(n)$ is the true optimal path cost from n to a goal.
2. **Consistency (Monotonicity):** A heuristic function $h(n)$ is consistent if, for every node n and every successor n' generated by action a , the estimated cost of reaching the goal from n is no greater than the step cost of reaching n' plus the estimated cost from n' :

$$h(n) \leq c(n, a, n') + h(n')$$

Every consistent heuristic is also admissible.



3.4.2 Greedy Best-First Search

Greedy best-first search expands the node with the lowest heuristic value $h(n)$, prioritizing nodes that appear closest to the goal state. The frontier is structured as a priority queue ordered strictly by $h(n)$.

While greedy best-first search can rapidly reach goal states in favorable environments, it is neither complete nor optimal. The algorithm can be led down inefficient or infinite paths if the heuristic function provides misleading estimates.

3.4.3 A* Search

A* search evaluates nodes by combining the accumulated cost $g(n)$ to reach node n and the heuristic cost estimate $h(n)$ to reach the goal from node n :

$$f(n) = g(n) + h(n)$$

The value $f(n)$ represents the estimated total cost of the path passing through node n from the start state to the goal state. A* search expands the node in the frontier with the lowest $f(n)$ value using a priority queue.

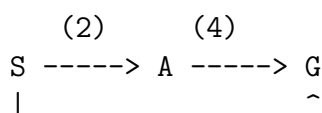
A* search is complete and optimal under the following conditions:

- When searching a tree structure, A* is optimal if $h(n)$ is admissible.
- When searching a graph structure (where duplicate paths to the same state are eliminated), A* is optimal if $h(n)$ is consistent.

3.4.4 Worked Example: Trace of A* Search

Consider a search problem on a directed graph with start node S and goal node G . The step costs and heuristic values $h(n)$ are given as follows:

- Edges and costs: $c(S, A) = 2$, $c(S, B) = 5$, $c(A, G) = 4$, $c(B, G) = 1$.
- Heuristic values: $h(S) = 5$, $h(A) = 3$, $h(B) = 1$, $h(G) = 0$.



| (5) (1) |
+-----> B -----+

The execution trace of A* graph search proceeds as follows:

1. **Initialization:** Place root S in frontier. $g(S) = 0$, $h(S) = 5$, $f(S) = 0 + 5 = 5$.
2. **Step 1:** Pop S ($f = 5$). Expand S to yield successors A and B :
 - Node A : $g(A) = 0 + 2 = 2$, $h(A) = 3$, $f(A) = 2 + 3 = 5$.
 - Node B : $g(B) = 0 + 5 = 5$, $h(B) = 1$, $f(B) = 5 + 1 = 6$.
Frontier contains: $\{A(f = 5), B(f = 6)\}$.
3. **Step 2:** Pop A ($f = 5$, lowest). Expand A to yield successor G :
 - Node G (via A): $g(G) = 2 + 4 = 6$, $h(G) = 0$, $f(G) = 6 + 0 = 6$.
Frontier contains: $\{B(f = 6), G(f = 6)\}$.
4. **Step 3:** Pop B ($f = 6$). Expand B to yield successor G :
 - Node G (via B): $g(G) = 5 + 1 = 6$, $h(G) = 0$, $f(G) = 6 + 0 = 6$. Since cost is identical to existing G , no update needed.
Frontier contains: $\{G(f = 6)\}$.
5. **Step 4:** Pop G ($f = 6$). Goal test passes. Return optimal path $S \rightarrow A \rightarrow G$ with cost 6.

3.5 Complexity Analysis and Combinatorial Explosion

Evaluating search algorithms requires analyzing their resource consumption in terms of time complexity (number of nodes generated) and space complexity (maximum number of nodes stored in memory simultaneously).

3.5.1 Complexity Parameters

Complexity is expressed using three structural parameters of the state space:

- b : The branching factor, defined as the maximum number of successors for any node.
- d : The depth of the shallowest optimal goal node.
- m : The maximum depth of any path in the state space (can be infinite).

3.5.2 Combinatorial Explosion

Combinatorial explosion describes the exponential growth in the number of states as search depth or branching factor increases. In a uniform tree with branching factor b , the total number of nodes generated up to depth d is:

$$N(b, d) = 1 + b + b^2 + b^3 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1}$$

If $b = 10$, a search tree of depth $d = 6$ contains 1,111,111 nodes. Increasing the depth to $d = 12$ expands the space to over 10^{12} nodes, consuming hundreds of gigabytes of RAM and substantial CPU runtime. As d grows further, exhaustive search becomes computationally intractable on physical hardware.

Algorithm	Complete?	Optimal?	Time Complexity	Space Complexity
Breadth-First Search	Yes	Yes ¹	$O(b^d)$	$O(b^d)$
Depth-First Search	No	No	$O(b^m)$	$O(b \cdot m)$
Uniform-Cost Search	Yes	Yes	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
Iterative Deepening	Yes	Yes ²	$O(b^d)$	$O(b \cdot d)$
Greedy Best-First	No	No	$O(b^m)$ worst case	$O(b^m)$ worst case
A* Search	Yes	Yes ³	Exponential worst case	Exponential worst case

Table 3.2: Comparison of Search Algorithm Properties

To mitigate combinatorial explosion, AI systems apply heuristic functions to prune unpromising branches, employ memory-bounded search variants (such as IDA* or SMA*), or utilize domain constraints to reduce the effective branching factor b .

The principles of systematic state evaluation and heuristic guidance introduced in this chapter establish the groundwork for knowledge representation, automated inference, and decision-making mechanisms examined in subsequent topics.

Tutorial Questions

1. Define the five structural components of a formal search problem. Formulate a complete search problem for a grid-based vacuum cleaner agent attempting to clean two dirty locations in a 2×2 grid.
2. Contrast uninformed search with informed search. Explain why Breadth-First Search requires exponential memory space relative to solution depth, whereas Depth-First Search requires linear space.
3. Explain the mechanics of Iterative Deepening Search. Demonstrate why the redundant generation of upper-level nodes in Iterative Deepening Search does not degrade its overall asymptotic time complexity compared to Breadth-First Search.
4. Define heuristic admissibility and heuristic consistency. Prove that every consistent heuristic function is admissible, and construct a counterexample or explanation showing why an admissible heuristic is not necessarily consistent.
5. Consider a search problem with start node S and goal G . The available actions, costs, and heuristic estimates $h(n)$ are defined as follows:
 - Step costs: $c(S, A) = 1$, $c(S, B) = 4$, $c(A, B) = 2$, $c(A, G) = 6$, $c(B, G) = 1$.

- Heuristics: $h(S) = 5$, $h(A) = 3$, $h(B) = 1$, $h(G) = 0$.

Show the step-by-step trace of A* graph search, listing the frontier contents, evaluation scores $f(n) = g(n) + h(n)$, and expanded nodes at each iteration. Identify the final path and total cost.

6. Calculate the total number of search nodes generated by Breadth-First Search in a state space with a uniform branching factor $b = 4$ up to depth $d = 5$. Compute the memory overhead if storing each search node requires 128 bytes.
7. Define the term combinatorial explosion. Discuss three distinct strategies used in software engineering and artificial intelligence to prevent combinatorial explosion from exceeding physical system memory during search operations.

Chapter 4

Knowledge Representation and Reasoning

An intelligent agent cannot act rationally in a complex environment without possessing an internal model of that environment. While earlier chapters explored search strategies that traverse predefined state spaces, automated decision making in real-world scenarios requires an agent to represent abstract entities, infer unobserved facts, and maintain a structured body of knowledge. This chapter examines how knowledge is formalized, stored, and manipulated within computer systems to enable sound automated reasoning.

Knowledge representation serves as the theoretical and algorithmic bridge between raw data perception and high-level reasoning. By mapping domain objects, properties, and relationships into precise formal structures, an artificial intelligence system can apply inference rules to answer queries, prove theorems, and make decisions under certainty or incomplete information.

This chapter details formal logic systems, theorem-proving mechanisms, methods for handling non-deterministic domains, and structured object-oriented representations. Mastering these representation schemes provides the foundation for designing rule-based Expert systems, understanding computational semantics in Natural Language Processing, and constructing autonomous planning architectures.

4.1 Fundamentals and Challenges of Knowledge Representation

4.1.1 Defining Knowledge Representation

Knowledge representation is the formal study and implementation of symbolic formalisms used to encode facts, rules, and relationships about the world so an automated agent can infer new knowledge. A knowledge representation system consists of two primary components: a formal language (with well-defined syntax and semantics) used to express propositions, and an inference engine that applies automated deduction procedures to those propositions to generate valid conclusions.

Knowledge is distinct from raw data or simple information. Data consists of unprocessed signals or numbers, such as temperature sensor values. Information organizes raw data into structured contexts, such as a log recording hourly temperature readings. Knowledge incorporates context, conditional rules, operational relationships, and cognitive constraints, allowing an agent to infer that a sudden temperature spike indicates a hardware malfunction.

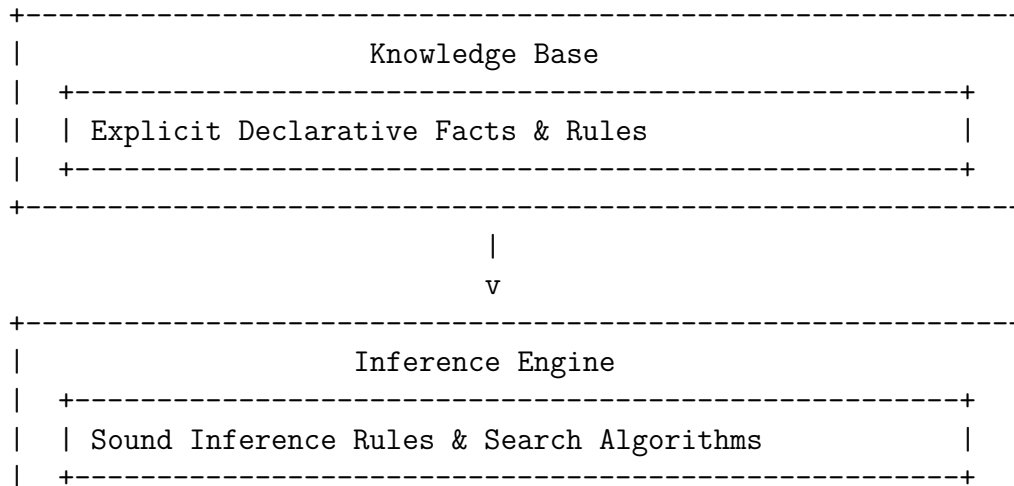
4.1.2 Core Challenges in Knowledge Representation

Designing an effective knowledge representation mechanism involves overcoming several fundamental computational and logical trade-offs. The primary challenges include:

- **Representational Adequacy:** The capability of a formal system to express every kind of knowledge needed for a specific problem domain.
- **Inferential Adequacy:** The ability of the inference mechanism to manipulate initial representation structures to yield new structures corresponding to valid conclusions.
- **Inferential Efficiency:** The capacity of the system to direct its computational resources toward the most promising paths during automatic inference, minimizing search complexity.
- **Acquisitional Efficiency:** The ease with which new knowledge can be inserted, updated, or extracted from the knowledge base without creating contradictions or systemic overhead.
- **Handling Incompleteness and Uncertainty:** The formal challenge of enabling a system to make decisions when domain knowledge is partially observable, noisy, or dynamic.

4.1.3 Knowledge Representation Languages and Paradigms

Knowledge representation languages are broadly categorized into formal declarative languages and structured procedural languages. Declarative representations state explicit facts and assertions about the world, leaving the deduction strategy to an independent inference engine. Structured and procedural representations embed operational knowledge directly into graph structures or algorithmic routines.



+-----+-----+-----+		
	v	
+-----+-----+-----+		
	Derived Inferences	
	+-----+-----+-----+	
	New Explicitly Proven Facts	
	+-----+-----+-----+	
+-----+-----+-----+		

4.2 Predicate Logic and Automated Theorem Proving

4.2.1 First-Order Predicate Logic

Predicate logic (specifically First-Order Predicate Logic, or FOL) extends propositional logic by introducing terms, predicates, variables, functions, and quantifiers. While propositional logic can only represent complete statements as atomic propositions, predicate logic allows internal structures of statements to be modeled accurately.

The formal syntax of predicate logic contains the following building blocks:

- **Constants:** Symbols that denote specific individual objects in the domain of discourse (e.g., *John*, *Osun*, 3).
- **Variables:** Symbols that range over individuals in the domain (e.g., x , y , z).
- **Functions:** Mappings from domain tuples to single domain objects (e.g., *fatherOf*(x)).
- **Predicates:** Relations defined over domain objects that evaluate to either True or False (e.g., *IsStudent*(x), *Greater*(x , y)).
- **Logical Connectives:** Symbols used to form complex expressions, including conjunction ($\wedge$), disjunction ($\vee$), negation ($\neg$), implication ($\rightarrow$), and equivalence ($\leftrightarrow$).
- **Quantifiers:** Universal quantification ($\forall x$, meaning "for all x ") and Existential quantification ($\exists x$, meaning "there exists an x ").

An expression in predicate logic is syntactically well-formed if terms and predicates are constructed according to these formal rules. For example, the assertion "Every computer science student takes an algorithm course" can be represented in predicate logic as:

$$\forall x(\text{CSStudent}(x) \rightarrow \exists y(\text{AlgorithmCourse}(y) \wedge \text{Takes}(x, y)))$$

4.2.2 Conversion to Clause Form

Automated inference algorithms such as resolution cannot easily operate on arbitrary predicate logic sentences containing mixed quantifiers and implication symbols. Before applying resolution, all logical statements must be converted into **Clause form** (also known as Conjunctive Normal Form). A clause is a disjunction (OR) of literals, and a complete knowledge base in Clause form is a conjunction (AND) of such clauses.

The conversion of a First-Order Predicate Logic formula into Clause form follows a deterministic six-step algorithm:

1. **Eliminate Implications and Equivalences:** Replace all occurrences of $P \rightarrow Q$ with $\neg P \vee Q$, and $P \leftrightarrow Q$ with $(\neg P \vee Q) \wedge (\neg Q \vee P)$.
2. **Reduce Scopes of Negation:** Push negation operators inward using De Morgan's laws ($\neg(P \wedge Q) \equiv \neg P \vee \neg Q$, $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$) and quantifier equivalences ($\neg\forall x P(x) \equiv \exists x \neg P(x)$, $\neg\exists x P(x) \equiv \forall x \neg P(x)$). Eliminate double negations ($\neg\neg P \equiv P$).
3. **Standardize Variable Names:** Rename bound variables so that each quantifier uses a unique variable name, preventing variable capture during subsequent transformations.
4. **Skolemize Existential Quantifiers:** Remove existential quantifiers ($\exists$). If an existential quantifier lies outside all universal quantifiers, replace its variable with a fresh Skolem constant. If it lies within the scope of universal quantifiers, replace the variable with a Skolem function of those universally quantified variables.
5. **Drop Universal Quantifiers:** Since all remaining variables are implicitly universally quantified, explicitly remove all universal quantifiers ($\forall$).
6. **Distribute Disjunctions over Conjunctions:** Convert the expression into Conjunctive Normal Form using the distributive law $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$. Separate the resulting conjunction into isolated clauses.

+-----+		
	First-Order Logic Sentence	
+-----+		
	v	
	1. Eliminate Implications & Equivalences	
	v	
	2. Reduce Scopes of Negation (De Morgan's Laws)	
	v	
	3. Standardize Variable Names	
	v	
	4. Skolemize Existential Quantifiers	
	v	
	5. Drop Universal Quantifiers	
	v	
	6. Distribute Disjunctions over Conjunctions	
	v	
+-----+		
	Clause Form	

+-----+

Worked Example: Clause Form Conversion

Convert the following sentence into Clause form: "Every human who has a parent is cared for by that parent."

$$\forall x \forall y ((\text{Human}(x) \wedge \text{ParentOf}(y, x)) \rightarrow \text{CaresFor}(y, x))$$

Step 1: Eliminate implication.

$$\forall x \forall y (\neg(\text{Human}(x) \wedge \text{ParentOf}(y, x)) \vee \text{CaresFor}(y, x))$$

Step 2: Reduce scope of negation.

$$\forall x \forall y (\neg \text{Human}(x) \vee \neg \text{ParentOf}(y, x) \vee \text{CaresFor}(y, x))$$

Step 3: Standardize variables (variables x and y are already distinct).

Step 4: Skolemize (there are no existential quantifiers).

Step 5: Drop universal quantifiers.

$$\neg \text{Human}(x) \vee \neg \text{ParentOf}(y, x) \vee \text{CaresFor}(y, x)$$

Step 6: Distribute disjunctions. The expression is already a single clause:

$$\{\neg \text{Human}(x), \neg \text{ParentOf}(y, x), \text{CaresFor}(y, x)\}$$

4.2.3 The Resolution Refutation Principle

Resolution is an inference rule for clausal logic that combines two clauses containing complementary literals to produce a single resolved clause, forming the basis of automated theorem proving. Two literals are complementary if one is the exact negation of the other.

The basic propositional resolution rule states that given two clauses $(A \vee C)$ and $(\neg C \vee B)$, an automated system can derive the resolvent clause $(A \vee B)$:

$$\frac{A \vee C, \quad \neg C \vee B}{A \vee B}$$

In First-Order Predicate Logic, complementary literals may contain variables. To resolve them, the engine must find a **unifier**, which is a substitution set θ that makes the atomic components of two literals syntactically identical.

Automated resolution typically proceeds via refutation (proof by contradiction):

1. Negate the goal statement that needs to be proven.
2. Convert the negated goal and all assertions in the knowledge base into Clause form.
3. Repeatedly apply resolution to selected pairs of clauses using unification.
4. Continue resolving until an empty clause ($\square$), representing an explicit logical contradiction, is derived.

Worked Example: Proof by Refutation Resolution

Given the following knowledge base:

1. All humans are mortal: $\forall x(\text{Human}(x) \rightarrow \text{Mortal}(x))$
2. Socrates is human: $\text{Human}(\text{Socrates})$

Goal: Prove that Socrates is mortal ($\text{Mortal}(\text{Socrates})$).

Step 1: Convert original facts to Clause form:

- Clause 1 (C_1): $\neg\text{Human}(x) \vee \text{Mortal}(x)$
- Clause 2 (C_2): $\text{Human}(\text{Socrates})$

Step 2: Negate the goal and add to clause set:

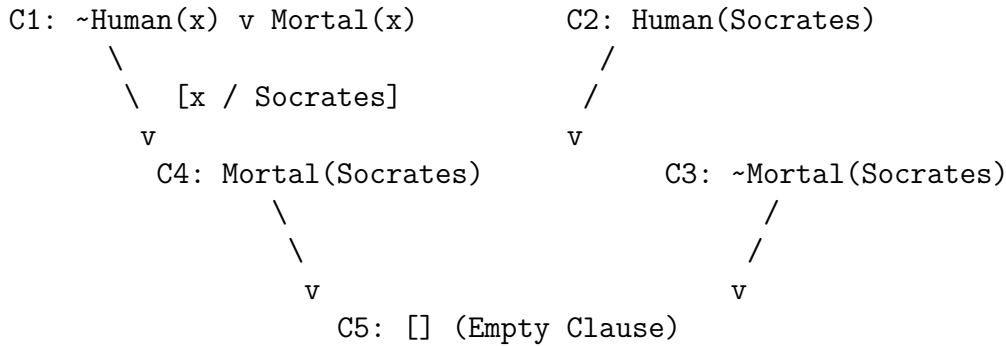
- Clause 3 (C_3): $\neg\text{Mortal}(\text{Socrates})$

Step 3: Resolve C_1 and C_2 . Substitute $\theta = \{x/\text{Socrates}\}$ into C_1 :

- Resolvent (C_4): $\text{Mortal}(\text{Socrates})$

Step 4: Resolve C_4 and C_3 :

- Resolvent (C_5): $\square$ (Empty Clause / Contradiction)



Because the negated goal produces a contradiction with the knowledge base, the original goal $\text{Mortal}(\text{Socrates})$ is formally proved.

4.3 Reasoning Under Uncertainty and Incompleteness

4.3.1 Non-Monotonic Logic

Classical logic systems are monotonic. In a monotonic logic system, if a set of axioms K entails a conclusion C , then adding any new axiom A to K will never invalidate C :

$$\text{If } K \models C \text{ then } (K \cup \{A\}) \models C$$

However, real-world commonsense reasoning operates under incomplete knowledge. Humans routinely make default assumptions that may be retracted when new evidence becomes available. **Non-monotonic logic** is a formal logic in which the set of provable conclusions can decrease when new axioms or facts are added to the knowledge base.

Non-monotonic logic resolves the qualification problem, which addresses the impossibility of explicitly enumerating all preconditions for an action or assertion. For instance, consider the standard default inference: "If x is a bird, assume x can fly." If the knowledge base learns that Tweety is a bird, it infers that Tweety flies. If the knowledge base is subsequently updated with the fact that Tweety is a penguin, the previous conclusion that Tweety flies is retracted.

4.3.2 Probabilistic Reasoning

While non-monotonic logic handles changing assumptions through default rules, **Probabilistic reasoning** provides a numerical framework for automated reasoning under uncertainty using probability theory to quantify belief states and update hypotheses given new evidence.

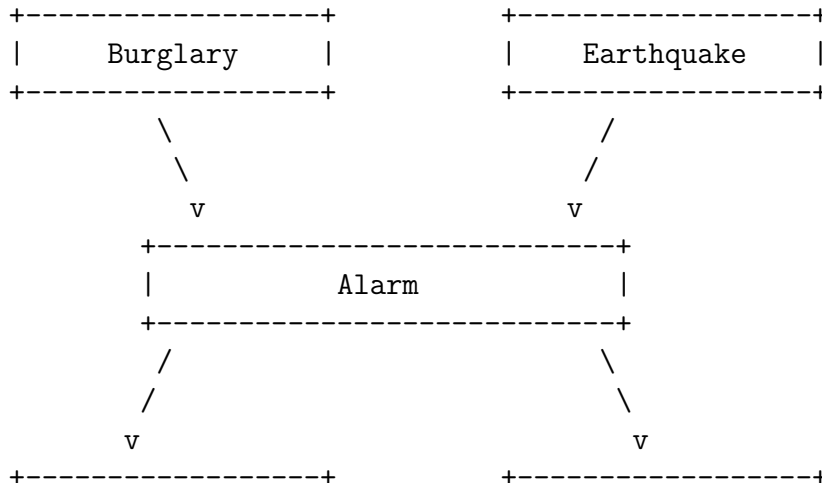
In a probabilistic domain, an agent represents its belief in a proposition H (hypothesis) using a probability distribution bounded between 0 and 1. When new evidence E is observed, the agent updates its belief using Bayes' Theorem:

$$P(H | E) = \frac{P(E | H)P(H)}{P(E)}$$

where:

- $P(H | E)$ is the posterior probability of hypothesis H given evidence E .
- $P(E | H)$ is the likelihood of observing evidence E assuming H is true.
- $P(H)$ is the prior probability of H .
- $P(E)$ is the marginal probability of the evidence E .

To compute belief states over complex domains efficiently, probabilistic reasoning uses Bayesian Belief Networks. A Bayesian Belief Network is a directed acyclic graph where nodes represent random variables and directed edges represent conditional causal dependencies.



John Calls	Mary Calls
------------	------------

4.4 Structured Knowledge Representations

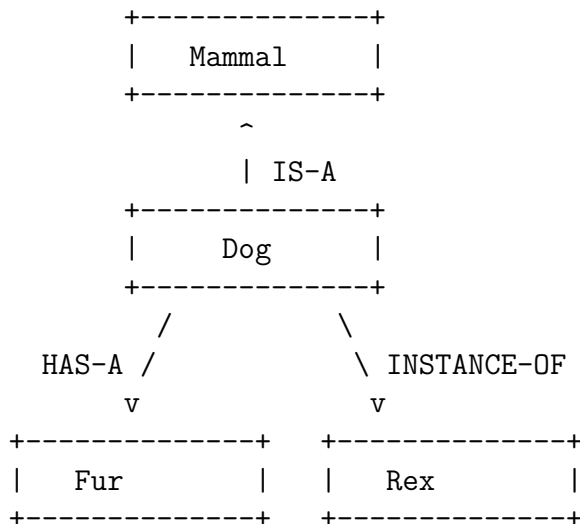
4.4.1 Semantic Networks

A **Semantic network** is a graphic data structure consisting of nodes representing concepts or objects and directed labeled arcs representing binary relations between them. Semantic networks organize domain knowledge into associative hierarchies, allowing properties to be inherited along taxonic edges.

The most common structural relations in a semantic network are:

- **IS-A**: Indicates subset relations between classes (e.g., Dog IS-A Mammal).
- **INSTANCE-OF**: Indicates membership of an individual object in a class (e.g., Rex INSTANCE-OF Dog).
- **HAS-A**: Indicates structural composition or attributes (e.g., Mammal HAS-A Fur).

Property inheritance enables a node to inherit attributes from higher-level class nodes automatically, eliminating repetitive storage across individual entries.



4.4.2 Frames

A **Frame** is a structured data format for organizing knowledge about a prototypical concept or situation using named attributes called slots and associated values called fillers. Formulated to capture structured expectations about complex entities, frames generalize semantic networks into object-oriented structures.

A frame slot can hold explicit atomic values, pointers to other frames, default values, or attached procedures:

- **Default Fillers:** Assumed attribute values used in the absence of explicit specific information.
- **Procedural Attachments:** Executable code attached to slots that trigger automatically. Common daemons include `if-needed` (executes when a slot value is read), `if-added` (executes when a value is inserted), and `if-removed` (executes when a value is modified or deleted).

A frame instance inherits slots and default values from its parent class frames unless explicitly overridden.

Structural Representation of a Frame

The structural representation of a system frame hierarchy for academic administration is shown below:

Frame: UniversityMember

IS-A: Top Concept

University: Osun State University

Status: Active

Frame: Student

IS-A: UniversityMember

Level: 300

TuitionPaid: False (Default)

If-Needed: CheckPaymentStatusProcedure

Frame: CSCStudent

IS-A: Student

Department: Computer Science

Major: Artificial Intelligence

EnrolledCourse: CSC 309

4.4.3 Comparison of Representation Schemes

Selecting a knowledge representation scheme involves balancing expressiveness, formal soundness, and algorithmic complexity. Table 4.1 summarizes the comparative properties of the formalisms covered in this chapter.

4.5 Conclusion

Knowledge representation schemes provide the foundational infrastructure that transforms raw symbols into actionable machine intelligence. By employing formal models such as predicate logic, Clause form resolution, non-monotonic default rules, probabilistic belief networks, and structured object frameworks, software engineers can design reasoning algorithms that operate predictably over domain contexts.

Table 4.1: Comparison of Knowledge Representation Formalisms

Formalism	Expressive Power	Primary Strengths	Main Limitations
Predicate Logic	Very High	Rigorous semantics, sound and complete proof theory	High computational complexity, brittle with uncertain facts
Non-Monotonic Logic	High	Handles default assumptions and retracts conclusions	Non-deterministic inference pathways, higher search complexity
Probabilistic Reasoning	Moderate to High	Quantifies uncertainty cleanly using likelihoods	Requires initial joint probability distribution estimates
Semantic Networks	Moderate	Intuitive visual layout, efficient inheritance processing	Lacks standard formal semantics, limited logical connectives
Frames	High	Object-oriented grouping, procedural attachments	Complex consistency enforcement across deep hierarchies

The representations studied in this chapter lead directly into specialized application fields. Expert systems rely on explicit declarative representations to emulate human expertise, while natural language understanding requires semantic networks and predicate structures to parse linguistic assertions. The next chapter applies these representation principles to the domain of Natural Language Processing.

Tutorial Questions

1. Explain the fundamental differences between data, information, and knowledge in automated computing systems.
2. Compare declarative knowledge representations with procedural knowledge representations. State two advantages and two disadvantages of each.
3. Convert the following predicate logic sentences into Clause form, detailing every step of the transformation process:
 - (a) $\forall x(\text{Student}(x) \rightarrow \exists y(\text{Book}(y) \wedge \text{Reads}(x, y)))$
 - (b) $\neg \exists x(\text{Corrupted}(x) \wedge \forall y(\text{File}(y) \rightarrow \text{Accesses}(x, y)))$
4. Given the following set of logical statements:
 - Every computer science student takes CSC 309.
 - Tunde is a computer science student.

Formulate these assertions in predicate logic, convert them into Clause form, and apply refutation resolution to prove that Tunde takes CSC 309.

5. Differentiate between monotonic logic and Non-monotonic logic. Describe a scenario in automated medical diagnosis where non-monotonic reasoning is necessary.
6. Define the term *procedural attachment* in the context of frame-based representations. Explain the operational differences between an **if-needed** daemon and an **if-added** daemon.
7. Construct an ASCII semantic network diagram representing a university hierarchy containing at least four distinct nodes and three relation types (**IS-A**, **INSTANCE-OF**, **HAS-A**). Explain how inheritance functions within your diagram.

Chapter 5

Natural Language Processing

Human communication relies primarily on natural language, which is inherently flexible, ambiguous, and context-dependent. **Natural Language Processing** (NLP) is the branch of artificial intelligence and computational linguistics that provides machines with the capacity to interpret, manipulate, and generate human language. This chapter examines the core principles of processing text and audio, covering structural linguistic levels, syntactic parsing, semantic representations, and practical applications such as question answering, sentiment analysis, machine translation, and speech recognition.

Developing computational models that handle natural language is crucial for modern computing systems. Unstructured textual and vocal data constitute a massive portion of human knowledge. Without automated language understanding, computer systems remain restricted to rigid, artificial command protocols. NLP enables scalable information extraction, automated user interaction, and seamless cross-lingual communication.

This chapter builds directly on the foundational concepts introduced earlier in the course. In Chapter 4, formal logical frameworks and symbolic structures were established to represent knowledge. NLP serves as the essential bridge that transforms raw, unstructured human language into these structured representations. The resulting structured knowledge can then be utilized by intelligent agents to reason about their environment or by expert systems to deliver specialized domain advice.

5.1 Introduction to Natural Language Processing and Understanding

Natural language processing encompasses a broad array of tasks ranging from simple string processing to complex conversational reasoning. To design effective AI systems, computational linguistic pipelines differentiate between basic text processing and deep conceptual comprehension.

5.1.1 Defining NLP and NLU

It is useful to distinguish between general processing and deep language understanding. **Natural Language Processing** refers to the overarching field encompassing any computational manipulation of natural language, including tokenization, word counting, part-of-speech tagging, and text formatting.

Natural Language Understanding (NLU) is a specialized subfield of NLP focused directly on machine comprehension. NLU systems extract the underlying meaning, intent, entities, and relational structure from natural language inputs. While basic NLP might compute word frequencies across a document, NLU determines whether the document expresses a binding legal obligation, a request for information, or a customer complaint.

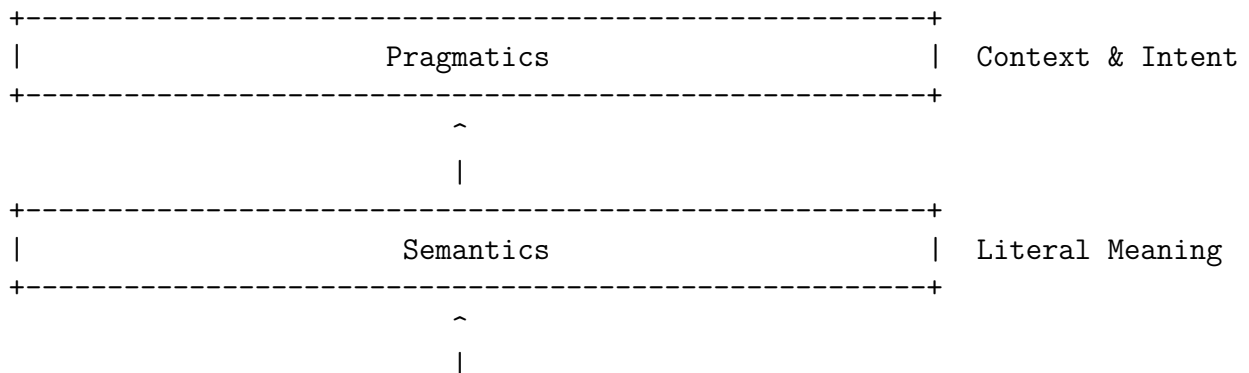
5.1.2 Key Challenges in Natural Language Processing

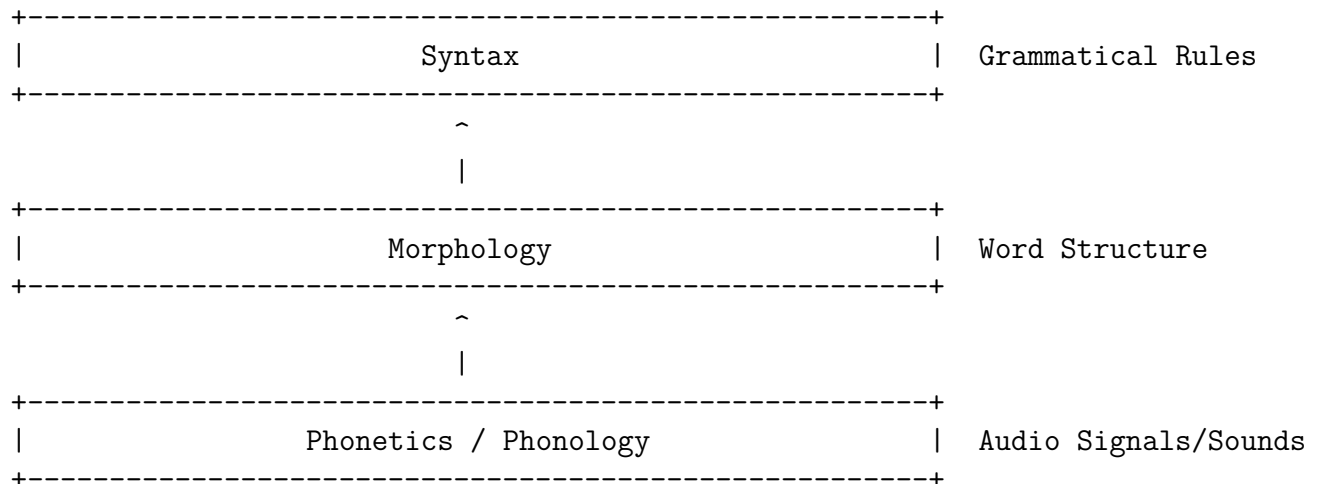
Automated processing of natural language is uniquely difficult due to characteristics inherent to human communication:

- **Ambiguity:** A single word, phrase, or sentence can have multiple distinct interpretations. Ambiguity occurs at lexical, syntactic, and semantic levels. For instance, the word “bank” can refer to a financial institution or the side of a river.
- **Context Sensitivity:** The meaning of a statement often depends heavily on surrounding sentences, discourse history, user background, or situational circumstances.
- **Variability and Evolution:** Human languages possess vast vocabularies, regional dialects, colloquialisms, and structural variations. Furthermore, languages evolve continuously as new terminology emerges.
- **Implicature and Pragmatics:** Humans frequently imply information without stating it explicitly. Understanding sarcasm, metaphor, and indirect requests requires world knowledge beyond formal vocabulary definitions.

5.2 Syntactic and Semantic Structures

Language processing algorithms operate across multiple hierarchical levels of abstraction. Moving from raw signals to deep understanding requires analyzing phonetic, morphological, syntactic, and semantic properties.





5.2.1 Levels of Linguistic Analysis

A comprehensive natural language framework evaluates language across six distinct levels:

1. **Phonetics and Phonology:** The study of physical speech sounds, acoustic waveforms, and the theoretical sound patterns of language.
2. **Morphology:** The study of word formation and the structure of morphemes, which are the smallest units of language carrying meaning (such as prefixes, roots, and suffixes).
3. **Syntax:** The structural arrangement of words to form grammatically valid sentences according to formal rules.
4. **Semantics:** The literal meaning of words, phrases, and full sentences independent of situational context.
5. **Pragmatics:** How language is interpreted within a specific communicative context, including speaker intent and indirect speech acts.
6. **Discourse Analysis:** The structure and relational connections present across sequences of multiple sentences.

5.2.2 Syntactic Parsing and Context-Free Grammars

Syntactic parsing is the formal process of mapping a sequence of tokens into a structural representation, such as a parse tree, to demonstrate its grammatical validity according to a given grammar.

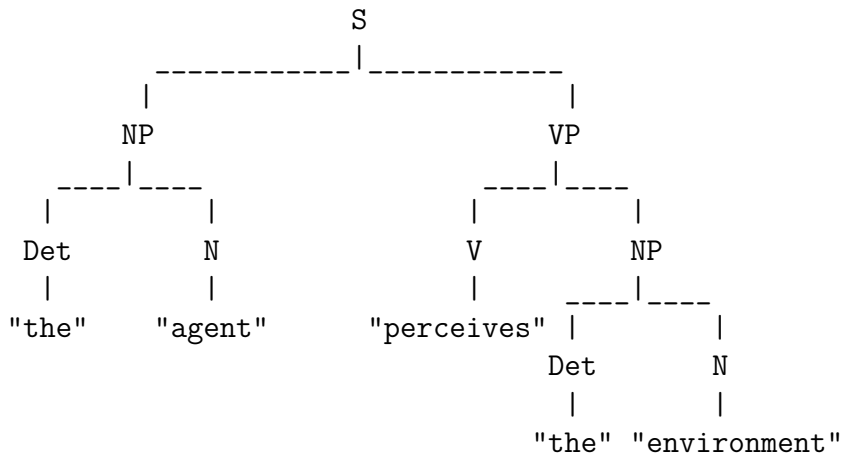
A standard formalism for specifying natural language syntax is the **Context-Free Grammar** (CFG). A CFG is formally defined as a four-tuple $G = (V, \Sigma, R, S)$, where:

- V is a finite set of non-terminal symbols (syntactic categories like NP for Noun Phrase, VP for Verb Phrase).
- Σ is a finite set of terminal symbols (the lexicon or actual words).
- R is a finite set of production rules mapping $V \rightarrow (V \cup \Sigma)^*$.
- $S \in V$ is the start symbol, usually representing a complete sentence (S).

Consider a simple grammar with production rules:

$$\begin{aligned}
 S &\rightarrow NP \quad VP \\
 NP &\rightarrow Det \quad N \\
 VP &\rightarrow V \quad NP \\
 Det &\rightarrow \text{"the"} \\
 N &\rightarrow \text{"agent"} \mid \text{"environment"} \\
 V &\rightarrow \text{"perceives"}
 \end{aligned}$$

Using this grammar, the input sentence “the agent perceives the environment” generates the following parse tree structure:



Syntactic parsing algorithms are classified into top-down techniques, which attempt to expand the start symbol S down to match the input terminals, and bottom-up techniques, which shift input terminals and reduce them into non-terminal constructs using production rules.

5.2.3 Semantic Analysis and Representation

Once a syntactic parse tree is constructed, semantic analysis translates the parse structure into a formal logic representation suitable for machine inference. This process relies on compositional semantics, where the meaning of a sentence is composed from the meanings of its syntactic subparts.

For instance, using **Predicate logic** introduced in Chapter 4, the sentence “The agent perceives the environment” maps to the formal relational predicate expression:

$$\text{Perceives}(\text{Agent}, \text{Environment})$$

This symbolic mapping enables the system to store the extracted statement inside a knowledge base, apply inference rules, and answer logical queries.

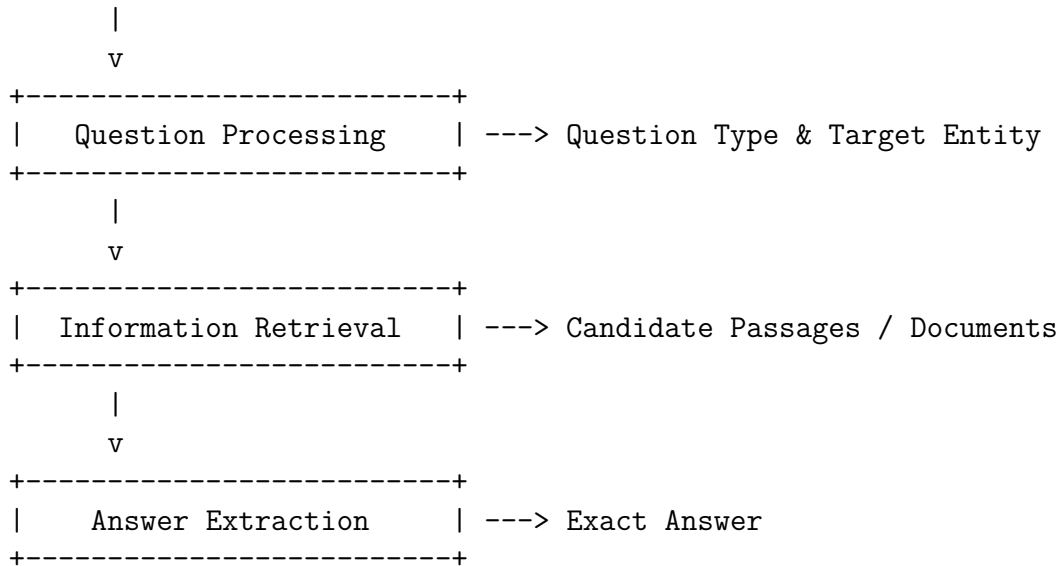
5.3 Question Answering Systems

Question Answering (QA) systems retrieve precise, concise answers to natural language questions posed by users, moving beyond simple key-word web search.

5.3.1 Architecture of Question Answering Systems

A classic pipeline QA system consists of three main modules: Question Processing, Information Retrieval (or Document Processing), and Answer Extraction.

User Question



5.3.2 Components of a Question Answering Pipeline

1. **Question Processing:** The input question is parsed to extract key keywords, formulate search queries, and classify the target answer type (such as Person, Location, Numerical Value, or Date).
2. **Information Retrieval:** The search query is issued to an indexed document collection or database to retrieve candidate documents or text passages that are likely to contain the answer.
3. **Answer Extraction:** Candidate passages undergo fine-grained semantic analysis, named entity recognition, and pattern matching. The system selects the exact text segment matching the target answer type and presents it to the user.

5.4 Sentiment Analysis

Sentiment analysis, also called opinion mining, is the computational task of identifying, extracting, and quantifying affective states and subjective polarities in text.

5.4.1 Levels of Granularity

Sentiment classification operates at three distinct structural levels:

- **Document Level:** Classifies the overall sentiment of an entire text document as positive, negative, or neutral.
- **Sentence Level:** Evaluates each sentence individually, determining whether it expresses subjective opinions and identifying its specific polarity.
- **Aspect Level:** Identifies specific entities or feature attributes within the text and assigns sentiment polarities to each. For example, in the review “The software interface is clear, but the execution speed is slow,” aspect-level analysis identifies a positive sentiment for “interface” and a negative sentiment for “execution speed.”

5.4.2 Methodological Approaches

Sentiment analysis primarily relies on two computational approaches:

- **Lexicon-Based Approaches:** These techniques utilize predefined dictionaries (lexicons) where individual words are pre-assigned sentiment scores. The overall text polarity is calculated by aggregating the individual scores of the words present. While simple and domain-independent, lexicon approaches struggle with negation, sarcasm, and domain-specific vocabulary.
- **Machine Learning Approaches:** Supervised machine learning algorithms (such as Naive Bayes, Support Vector Machines, or Neural Networks) train on labeled corpora. Sentences are converted into numerical feature vectors (such as n-gram counts or continuous word embeddings) to classify sentiment labels.

Table 5.1: Comparison of Sentiment Analysis Approaches

Feature	Lexicon-Based Approach	Machine Learning Approach
Training Requirements	None required; relies on static dictionaries.	Requires substantial labeled training data.
Domain Adaptation	Poor; dictionary terms are fixed.	High; retrain effectively on domain corpora.
Context Awareness	Low; often treats words independently.	High; captures context and word dependencies.
Computational Cost	Extremely low runtime overhead.	Requires training compute and inference power.

5.5 Machine Translation

Machine translation (MT) is the automated process of converting text or speech from a source natural language into a target natural language while preserving both content and grammatical structure.

5.5.1 Overview of Paradigms

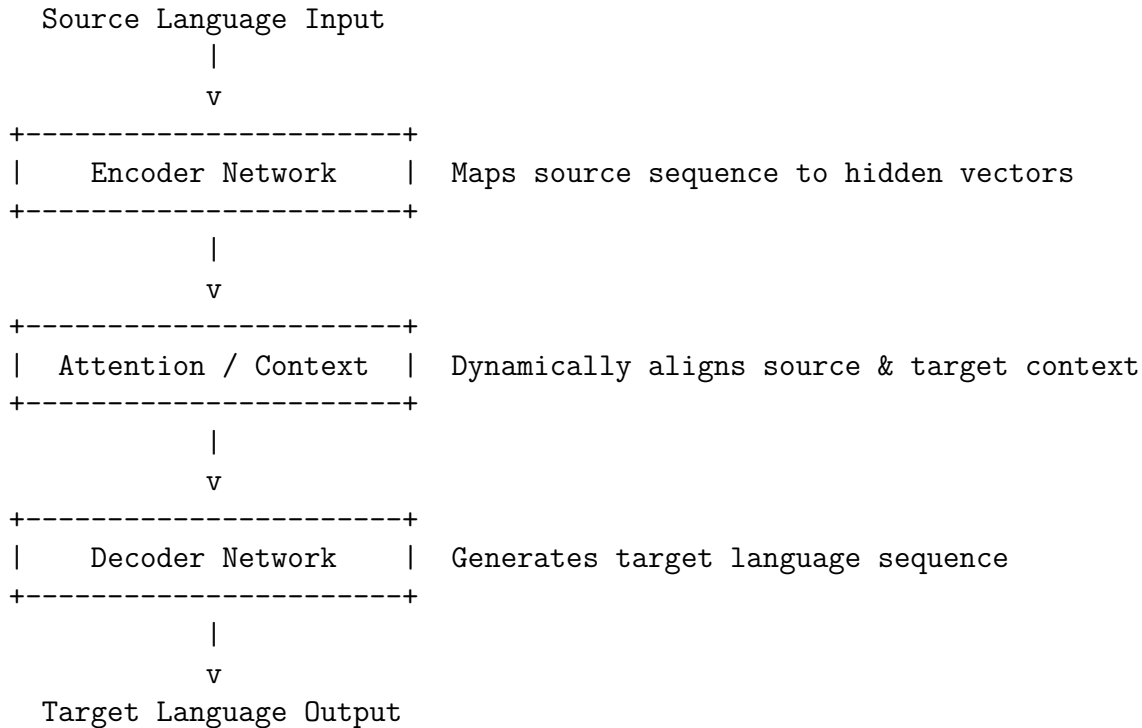
Machine translation paradigms have evolved through three major technological generations:

1. **Rule-Based Machine Translation (RBMT)**: RBMT systems depend on manually crafted grammatical rules, bilingual dictionaries, and structural transfer algorithms developed by expert linguists.
2. **Statistical Machine Translation (SMT)**: SMT systems analyze large bilingual parallel corpora to compute translation probabilities. SMT uses a translation model $P(F | E)$ to evaluate phrase correspondence alongside a language model $P(E)$ to enforce fluency in the target language:

$$\hat{E} = \arg \max_E P(F | E) P(E)$$

where F represents source language sentences and E represents candidate target language sentences.

3. **Neural Machine Translation (NMT)**: Modern NMT systems train end-to-end neural network architectures (such as Sequence-to-Sequence models with attention mechanisms or Transformer architectures). NMT constructs continuous vector representations of whole sentences, enabling fluent, contextually accurate translations.



5.6 Speech Recognition

Speech recognition, or Automatic Speech Recognition (ASR), converts continuous acoustic audio waveforms recorded by microphones into written textual transcriptions.

Table 5.2: Comparative Analysis of Machine Translation Paradigms

Dimension	Rule-Based (RBMT)	Statistical (SMT)	Neural (NMT)
Primary Resource	Human expert rules	Parallel bilingual corpora	Massive parallel corpora
Translation Fluency	Low; rigid output	Moderate; phrase fragments	High; highly natural fluency
System Complexity	Complex rule maintenance	High statistical parameter tuning	End-to-end deep learning
Out-of-Vocabulary	Handled by adding rules	Handled by backoff models	Handled via subword tokenization

5.6.1 Acoustic and Language Modeling

A classic speech recognition system relies on two primary probabilistic components working in conjunction:

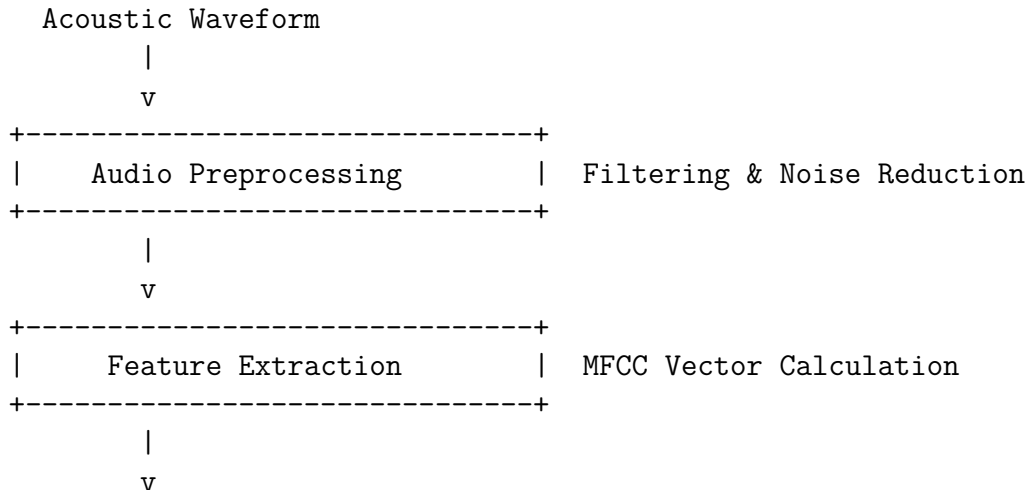
- **Acoustic Model:** Computes the probability $P(A | W)$ of observing a specific acoustic feature sequence A given a sequence of spoken words W . It maps physical audio signals (converted to spectral features like Mel-Frequency Cepstral Coefficients) to phonemes.
- **Language Model:** Computes the prior probability $P(W)$ of a target word sequence W occurring in a given language. It ensures that output transcriptions are syntactically and semantically plausible.

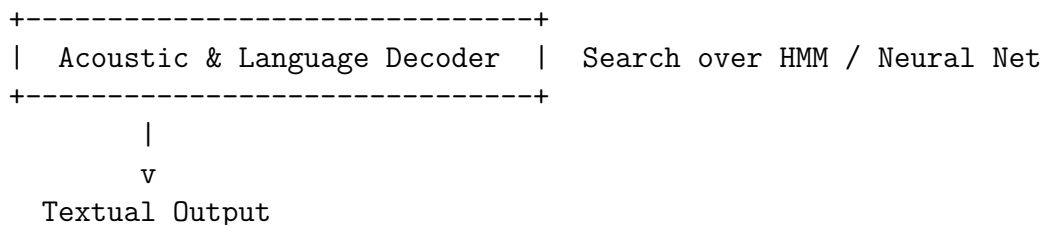
Applying Bayes' Rule, the decoder identifies the optimal word sequence $\hat{W}$:

$$\hat{W} = \arg \max_W P(A | W) P(W)$$

5.6.2 Speech-to-Text Processing Pipeline

The speech recognition process follows a structured sequential pipeline:





5.7 Summary

Natural language processing is a fundamental discipline within artificial intelligence, enabling machines to understand, evaluate, and generate human language. By structuring language into phonetic, morphological, syntactic, semantic, and pragmatic levels, AI systems transform raw textual and vocal inputs into actionable formal knowledge representations. Applications such as question answering, sentiment analysis, machine translation, and speech recognition demonstrate the practical capabilities of these frameworks.

In the next chapter, we will explore Expert Systems and AI Programming Languages. We will see how domain knowledge and derived semantic structures are formally codified into specialized rule bases, inference engines, and dedicated programming environments to solve complex domain problems.

Tutorial Questions

1. Distinguish clearly between Natural Language Processing (NLP) and Natural Language Understanding (NLU). Provide a practical computing example illustrating the difference.
2. Given the Context-Free Grammar rules:

$$\begin{aligned}
 S &\rightarrow NP \ VP \\
 NP &\rightarrow Det \ N \\
 VP &\rightarrow V \ NP \\
 Det &\rightarrow \text{"a"} \mid \text{"the"} \\
 N &\rightarrow \text{"robot"} \mid \text{"symbol"} \\
 V &\rightarrow \text{"generates"}
 \end{aligned}$$

Construct the corresponding parse tree for the sentence: "a robot generates the symbol".

3. Explain the six levels of linguistic analysis. Describe the primary objective of each level when building a natural language processing system.
4. Compare lexicon-based approaches and machine learning approaches to sentiment analysis across three criteria: data requirements, computational complexity, and contextual accuracy.
5. Outline the three main processing stages of a question answering pipeline. Explain why question classification is critical to the accuracy of answer extraction.
6. Describe the Statistical Machine Translation equation $\hat{E} = \arg \max_E P(F \mid E)P(E)$. Explain the specific roles played by the translation model $P(F \mid E)$ and the language model $P(E)$.

7. Explain how acoustic models and language models collaborate in an automatic speech recognition system to resolve ambiguous speech input.

Chapter 6

Expert Systems and AI Programming Languages

6.1 Introduction

Expert systems represent one of the earliest and most commercially successful operational paradigms in artificial intelligence. An **expert system** is a computer program designed to emulate the decision-making and problem-solving ability of a human expert within a specific, well-defined domain. Unlike conventional software systems that blend control flow logic and domain data into a single programmatic structure, expert systems establish a clean structural separation between domain knowledge and the inference engine that computes over that knowledge.

Studying expert systems and AI programming languages is essential for understanding how symbolic reasoning is applied to practical domain problems. Expert systems provide a practical framework for deploying domain rules, declarative facts, and automated inference to perform tasks such as medical diagnosis, mineral exploration, fault isolation, and financial auditing. Furthermore, examining dedicated AI programming languages highlights how theoretical representations direct computational implementation choices.

This chapter details the internal architecture, functional capabilities, and developmental phases of expert systems. It investigates the core inference algorithms of forward chaining and backward chaining, accompanied by detailed evaluation traces. Additionally, this chapter surveys traditional declarative languages, specifically Lisp and Prolog, alongside modern multi-paradigm languages such as Python and C++, evaluating their paradigms, memory mechanics, and suitability for artificial intelligence implementations.

6.2 Characteristics and Architecture of Expert Systems

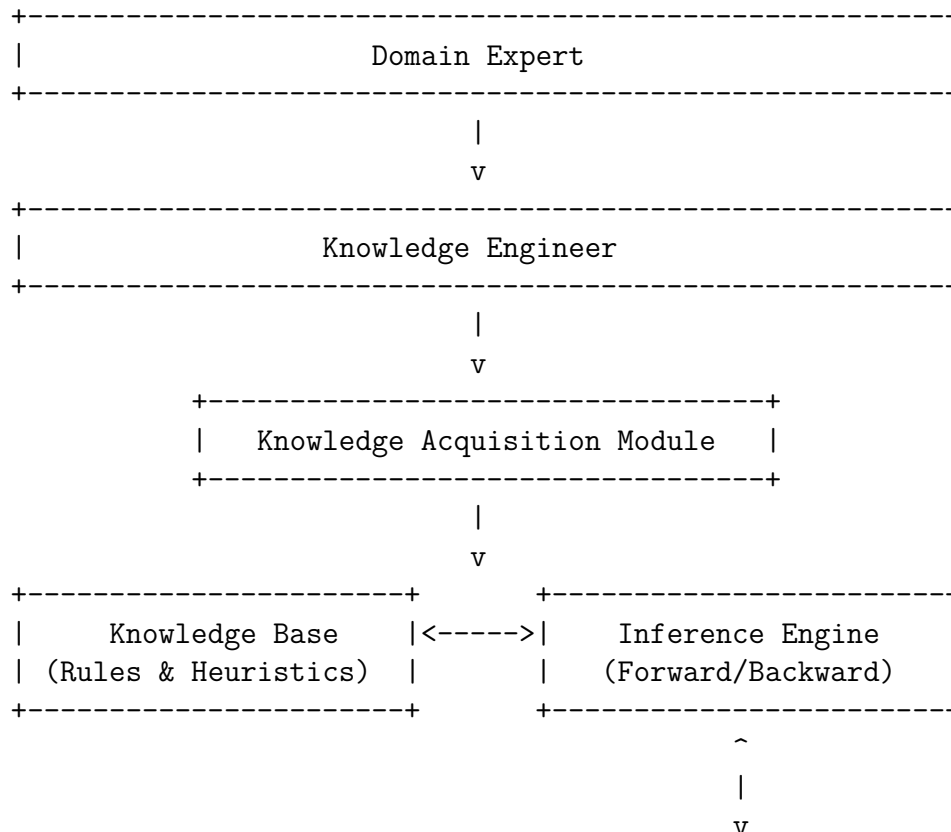
6.2.1 Core Characteristics

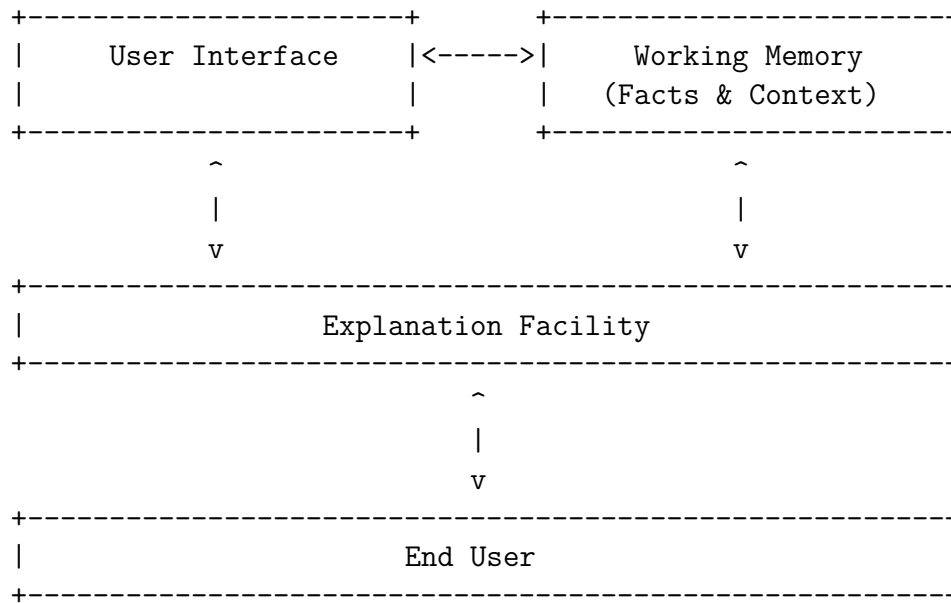
Expert systems possess distinct functional properties that distinguish them from standard algorithmic programs:

- **High Performance:** They deliver domain decisions comparable in quality to human specialists within narrow boundaries.
- **Domain Specificity:** They operate over constrained problem spaces containing explicit, formalizable expert rules rather than broad common-sense knowledge.
- **Explanation Capability:** They maintain an internal audit trail enabling them to explain the logical progression that produced a given conclusion.
- **Symbolic Reasoning:** They operate primarily on explicit symbols, propositions, and categorical assertions rather than purely numerical algorithms.
- **Separation of Knowledge and Control:** Domain facts and inference mechanisms reside in decoupled software layers, enabling modular knowledge updates without code refactoring.

6.2.2 Architectural Components

The operational structure of a standard expert system consists of several interacting software modules. Figure 1 illustrates this structural topology.





The primary architectural elements include:

1. **Knowledge Base:** A structured repository containing permanent domain knowledge represented as IF-THEN production rules, factual predicates, and domain-specific heuristics.
2. **Working Memory (Fact Base):** A dynamic temporary memory store that holds current facts, user inputs, and intermediate assertions deduced during an active reasoning session.
3. **Inference Engine:** The execution kernel that evaluates rules stored in the knowledge base against current assertions in working memory to infer new facts or reach conclusions.
4. **Explanation Facility:** A diagnostic subsystem that answers user queries concerning how a specific hypothesis was proved or why a particular question was asked.
5. **User Interface:** The communication interface through which the user inputs problem parameters and receives structural recommendations or diagnostic findings.
6. **Knowledge Acquisition Module:** An administrative subsystem that enables knowledge engineers to insert, modify, or validate rules within the knowledge base.

6.3 Inference Mechanisms: Forward and Backward Chaining

6.3.1 Production Rules and the Match-Resolve-Execute Cycle

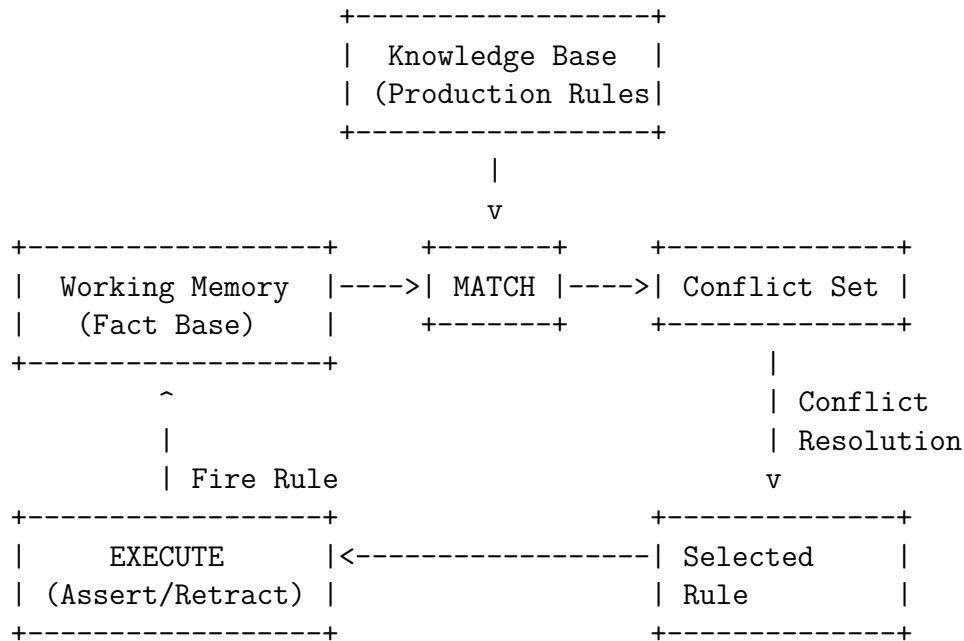
Knowledge within an expert system is predominantly expressed using production rules structured as:

IF ⟨antecedents⟩ THEN ⟨consequents⟩

When the antecedents evaluate to true given the contents of working memory, the rule becomes eligible to fire, asserting its consequents into working memory. The inference engine manages

rule evaluation through a three-phase cycle:

1. **Match:** Compare current facts in working memory against rule antecedents to build a conflict set of all satisfied rules.
2. **Conflict Resolution:** Apply a heuristic strategy (such as rule priority, recency, or specificity) to select a single rule from the conflict set.
3. **Execute (Fire):** Execute the action or assertion of the selected rule, updating working memory, and repeat the cycle.



6.3.2 Forward Chaining

Forward chaining is a data-driven reasoning strategy. The inference engine begins with a collection of known facts in working memory and continuously evaluates production rules to deduce new facts. This process iterates until no further rules can fire or a predefined goal state enters working memory.

Forward chaining is optimal for tasks involving monitoring, synthesis, data analysis, and open-ended planning, where initial data is available but specific goal states are not tightly defined in advance.

6.3.3 Backward Chaining

Backward chaining is a goal-driven reasoning strategy. The inference engine starts with a proposed target hypothesis (goal) and checks whether working memory contains facts that confirm it. If the hypothesis is unknown, the engine locates rules whose consequents match the goal and sets their antecedents as secondary sub-goals. This process continues recursively until all sub-goals are matched against known facts or resolved through direct user input.

Backward chaining is ideal for diagnostic systems, troubleshooting tasks, and verification problems, where a clear hypothesis exists and relevant facts must be sought selectively.

6.3.4 Comparison of Inference Strategies

Table 1 summarizes the operational structural differences between forward chaining and backward chaining.

Dimension	Forward Chaining	Backward Chaining
Primary Strategy	Data-driven (bottom-up)	Goal-driven (top-down)
Starting Point	Initial known facts	Proposed hypothesis or goal
Processing Direction	Moves from premises to conclusions	Moves from target goal to required premises
Conflict Resolution	Required when multiple rules match facts	Required when multiple rules produce same goal
Best Suited For	Monitoring, synthesis, planning, control	Diagnosis, debugging, classification, audit
Search Space	Can explore broad state spaces	Focused strictly on relevant hypothesis branches

Table 6.1: Comparison of Forward and Backward Chaining.

6.3.5 Worked Example: Diagnostic Trace

Consider a diagnostic rule base for a simplified network fault management expert system containing four production rules:

- **Rule 1:** IF link_down AND router_unreachable, THEN gateway_failure.
- **Rule 2:** IF gateway_failure AND dns_timeout, THEN network_outage.
- **Rule 3:** IF link_down AND high_packet_loss, THEN cable_fault.
- **Rule 4:** IF network_outage AND backup_link_inactive, THEN critical_alert.

Assume initial Working Memory contains: {link_down, router_unreachable, dns_timeout, backup_link_inactive}

Forward Chaining Trace

1. Cycle 1:

- Evaluate rule premises against Working Memory.
- Rule 1 premise (link_down $\wedge$ router_unreachable) is satisfied.
- Conflict set: {Rule 1}.
- Fire Rule 1: Assert gateway_failure.
- Updated Working Memory: {link_down, router_unreachable, dns_timeout, backup_link_inactive, gateway_failure}

2. Cycle 2:

- Evaluate rule premises against updated Working Memory.

- Rule 2 premise ($\text{gateway_failure} \wedge \text{dns_timeout}$) is satisfied.
 - Conflict set: $\{\text{Rule 2}\}$.
 - Fire Rule 2: Assert network_outage .
 - Updated Working Memory: $\{\text{link_down}, \text{router_unreachable}, \text{dns_timeout}, \text{backup_link_inactive}\}$.
3. **Cycle 3:**
- Evaluate rule premises against updated Working Memory.
 - Rule 4 premise ($\text{network_outage} \wedge \text{backup_link_inactive}$) is satisfied.
 - Conflict set: $\{\text{Rule 4}\}$.
 - Fire Rule 4: Assert critical_alert .
 - Updated Working Memory: $\{\dots, \text{critical_alert}\}$.
4. **Termination:** No additional rules match. System concludes critical_alert .

Backward Chaining Trace

Goal to verify: critical_alert .

1. Check if critical_alert is in Working Memory. Result: False.
2. Find rule with consequent critical_alert . Result: Rule 4.
3. Set premises of Rule 4 as sub-goals: Sub-goal A (network_outage), Sub-goal B ($\text{backup_link_inactive}$).
4. Evaluate Sub-goal B ($\text{backup_link_inactive}$): Present in Working Memory (TRUE).
5. Evaluate Sub-goal A (network_outage): Not in Working Memory. Locate rule concluding network_outage . Result: Rule 2.
6. Set premises of Rule 2 as sub-goals: Sub-goal A1 (gateway_failure), Sub-goal A2 (dns_timeout).
7. Evaluate Sub-goal A2 (dns_timeout): Present in Working Memory (TRUE).
8. Evaluate Sub-goal A1 (gateway_failure): Not in Working Memory. Locate rule concluding gateway_failure . Result: Rule 1.
9. Set premises of Rule 1 as sub-goals: Sub-goal A1a (link_down), Sub-goal A1b ($\text{router_unreachable}$).
10. Evaluate Sub-goal A1a (link_down): Present in Working Memory (TRUE).
11. Evaluate Sub-goal A1b ($\text{router_unreachable}$): Present in Working Memory (TRUE).
12. Satisfy Rule 1 premises $\implies$ Assert gateway_failure (TRUE).
13. Satisfy Rule 2 premises $\implies$ Assert network_outage (TRUE).
14. Satisfy Rule 4 premises $\implies$ Confirm Goal critical_alert is PROVED.

6.4 Expert System Technology and Development Lifecycle

6.4.1 Development Phases

Constructing an expert system requires a systematic lifecycle:

1. **Identification:** Define boundaries, domain requirements, resource constraints, and

explicit performance metrics.

2. **Conceptualization:** Elicit expert domain knowledge to identify key concepts, primitive entities, relations, and control strategies.
3. **Formalization:** Map domain concepts into formal computational models such as predicate logic, frame systems, or production rules.
4. **Implementation:** Program facts and production rules into an operational environment or expert system shell.
5. **Testing and Validation:** Evaluate generated outputs against historical test cases and human expert judgments to quantify accuracy.

6.4.2 Roles in System Development

Development relies on specialized multi-disciplinary roles:

- **Domain Expert:** A human specialist who possesses institutional domain knowledge, heuristics, and procedural techniques.
- **Knowledge Engineer:** An AI practitioner who interviews domain experts, extracts implicit domain heuristics, and translates them into formal representations.
- **System Developer:** A software engineer who builds user interfaces, integrates system components, and connects external database pipelines.
- **End User:** The operational user who interacts with the system to obtain domain decisions.

6.4.3 Expert System Shells

An **expert system shell** is a software framework containing an inference engine, user interface, and explanation facility, stripped of all domain-specific knowledge.

Shells eliminate the need to build reasoning components from scratch. Developers populate the empty shell with domain rules and facts, accelerating deployment. Standard historically significant shells include CLIPS, JESS, and EMYCIN.

6.4.4 Advantages and Limitations

Expert systems deliver clear operational advantages alongside systemic constraints:

- **Advantages:**
 - Permanent capture and distribution of scarce human expertise.
 - Continuous, objective, and unbiased decision execution.
 - Reduced operational cost and risk in hazardous physical environments.
 - Full auditing capability through explicit step-by-step trace facilities.
- **Limitations:**
 - **Knowledge Acquisition Bottleneck:** Extracting tacit human knowledge and formalizing it into discrete logical rules is difficult and slow.

- **Brittleness:** Systems fail completely when presented with edge cases outside their explicit rule set.
- **Lack of Common Sense:** They cannot reason outside their designated domain or leverage implicit worldly background knowledge.
- **High Maintenance Costs:** Large rule sets grow fragile over time, leading to unexpected rule interactions during updates.

6.5 Programming Languages for Artificial Intelligence

6.5.1 Overview of AI Programming Paradigms

Artificial intelligence development spans three core software paradigms:

1. **Declarative / Logic Paradigm:** Specifies *what* relationships and conditions hold rather than *how* to execute computations step by step.
2. **Functional Paradigm:** Structures computation as the evaluation of pure mathematical functions, avoiding mutable state and side effects.
3. **Imperative / Object-Oriented Paradigm:** Expresses algorithms as state changes, providing granular control over memory layout and processing cycles.

6.5.2 Lisp: List Processing and Symbolic Manipulation

Developed by John McCarthy in 1958, Lisp is one of the oldest high-level programming languages. It established fundamental paradigms for symbolic AI.

Key architectural features include:

- **Symbolic Data Structures:** Everything in Lisp is expressed using symbolic expressions (s-expressions), represented internally as linked tree structures.
- **Homoiconicity (Code-as-Data):** Lisp code shares the exact same representation format as Lisp data structures. This property enables programs to dynamically read, modify, compile, and generate other Lisp code.
- **Recursion and Memory Management:** Lisp relies on recursion for control structures and introduced automatic garbage collection to manage heap dynamic allocations.

The following code segment demonstrates recursive list search written in Lisp syntax:

```
(defun element-exists (item lst)
  (cond
    ((null lst) nil) ; Base case: empty list
    ((equal item (car lst)) t) ; Match found at head
    (t (element-exists item (cdr lst)))) ; Recursive call on tail
```

6.5.3 Prolog: Programming in Logic

Created in the early 1970s by Alain Colmerauer and Robert Kowalski, Prolog (Programming in Logic) is a declarative language based on first-order predicate logic and resolution theorem proving.

Key operational features include:

- **Declarative Assertions:** Programs consist of logical facts and conditional rules. The user initiates execution by firing a query.
- **Unification:** An automatic pattern-matching mechanism that binds terms and variables across premises.
- **Automatic Backtracking:** When a branch of a logical search fails, Prolog automatically unwinds state execution and explores alternative matching clauses in depth-first order.

The following Prolog program illustrates family relationship rules and query evaluation:

```
% Facts
parent(solomon, mary).
parent(mary, john).

% Rule
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

% Query
% ?- grandparent(solomon, john).
% Output: true.
```

6.5.4 Modern Languages in Contemporary AI: Python and C++

While Lisp and Prolog dominated early symbolic AI, modern artificial intelligence demands numerical optimization, tensor computations, and massive data parallelism.

- **Python:** Python is currently the dominant language for modern machine learning, deep learning, and data engineering. Its primary advantage lies in minimal syntactical overhead and a vast ecosystem of C-optimized computational libraries, including NumPy, PyTorch, TensorFlow, and Scikit-learn. However, as an interpreted language, native Python exhibits lower raw execution speed.
- **C++:** C++ remains the industry standard for high-performance execution environments, game engines, embedded robotics, and real-time vision processing. It offers explicit memory allocation control, compiled speed, and deterministic resource lifecycle management. Standard deep learning frameworks implement high-level APIs in Python while executing underlying neural network kernels in C++.

6.5.5 Comparison of AI Programming Languages

Table 2 compares the four primary languages across standard technical dimensions.

Property	Lisp	Prolog	Python	C++
Primary Paradigm	Functional / Symbolic	Declarative Logic	Multi-paradigm / Object-Oriented	Systems / Object-Oriented
Primary AI Area	Symbolic AI, Theorem Proving	Knowledge Bases, Expert Systems	Deep Learning, Data Science, NLP	Robotics, Real-time Vision, Embedded AI
Memory Management	Automatic Garbage Collection	Managed Engine Stack	Automatic Garbage Collection	Manual or Smart Pointers
Strengths	Homoiconicity, dynamic macros	Built-in inference and unification	Massive ecosystem, clear syntax	Direct memory access, top execution speed
Weaknesses	Non-standard syntax, steep learning curve	Inefficient for raw matrix math	Slower runtime execution speed	Complex syntax, manual safety burden

Table 6.2: Comparative Analysis of AI Programming Languages.

6.6 Conclusion

Expert systems demonstrate how domain knowledge can be formalized into production rules and evaluated using structured engines like forward and backward chaining. Programming languages such as Lisp and Prolog established symbolic manipulation and declarative inference, while modern multi-paradigm languages like Python and C++ drive large-scale, high-performance numerical AI applications. As system designs transition from discrete rules to continuous sensory interpretation, these architectural principles provide a foundation for complex interactive environments. The next chapter examines Computer Vision and Robotics, exploring how autonomous agents process visual input and act upon physical environments.

Tutorial Questions

1. Explain the primary structural difference between conventional software architectures and expert system architectures. Why is this separation advantageous for long-term system maintenance?
2. Define the terms *knowledge base*, *inference engine*, and *working memory*. Describe how these three components interact during an execution cycle.
3. Consider the following production rule base:
 - Rule 1: IF *A* AND *B*, THEN *C*.
 - Rule 2: IF *C* AND *D*, THEN *E*.

- Rule 3: IF E AND F , THEN G .

Given initial Working Memory containing facts $\{A, B, D, F\}$, trace the execution of a forward chaining inference engine step by step. Show the contents of Working Memory after each rule fires.

4. Using the rule base from Question 3, trace how a backward chaining inference engine proves the hypothesis goal G . Detail each sub-goal created and evaluated during search.
5. Compare forward chaining and backward chaining across the following dimensions: primary search direction, starting state, optimal use cases, and handling of broad goal spaces.
6. Describe what an expert system shell is. Explain how the use of a shell affects the system engineering process during software development.
7. What is meant by the term *knowledge acquisition bottleneck*? Identify two factors that contribute to this problem when building expert systems.
8. Define *homoiconicity* as exhibited by Lisp, and explain how it facilitates dynamic code generation. Compare Prolog's declarative logic execution model with Python's imperative evaluation model for artificial intelligence applications.

Chapter 7

Computer Vision and Robotics

Artificial intelligence systems must bridge the gap between abstract symbolic reasoning and physical reality. While prior chapters examined how intelligent agents represent knowledge, draw logical inferences, process human text, and execute search algorithms, an agent operating in the physical world requires capabilities to perceive its environment and execute physical actions. Perception and physical execution allow an agent to understand complex visual scenes and manipulate physical objects in real time.

This chapter explores two closely integrated fields: computer vision and robotics. Computer vision provides the sensory foundation, enabling software to convert raw multidimensional signal data from digital cameras into structured visual scene representations. Robotics provides the physical instantiation, enabling autonomous software agents to control mechanical structures, navigate physical terrain, and perform precision tasks.

Understanding these concepts is essential for completing the artificial intelligence paradigm. By unifying visual image recognition, facial analysis, kinematically driven movement, and software control architectures, intelligent agents transition from isolated software algorithms into fully embodied autonomous systems capable of acting effectively within real-world environments.

7.1 Computer Image Recognition

Computer vision is an interdisciplinary field of artificial intelligence and computer science concerned with enabling computational systems to acquire, process, analyze, and extract meaningful structural information from digital images or video streams. Unlike basic image processing, which transforms one image into another image (such as applying contrast enhancement), computer vision extracts high-level semantic descriptions and physical properties from visual data.

7.1.1 Image Representation and Preprocessing

A digital gray-scale image is represented mathematically as a two-dimensional matrix or discrete spatial function $I(x, y)$, where x and y denote spatial coordinates within a bounded grid. The value $I(x, y)$ represents the light intensity or brightness at that specific pixel location, typically quantized as an integer value ranging from 0 (pure black) to 255 (pure white) for an 8-bit representation. Color images extend this formulation by stacking three color channels, representing Red, Green, and Blue (RGB) intensity matrices:

$$I_{\text{color}}(x, y) = \begin{bmatrix} I_R(x, y) \\ I_G(x, y) \\ I_B(x, y) \end{bmatrix}$$

Raw sensory images captured by digital camera sensors often contain high-frequency acoustic or thermal noise, spatial blurring, and uneven illumination. Preprocessing techniques clean and prepare visual data for downstream interpretation. Linear spatial filtering reduces high-frequency noise by convolving the image matrix I with a spatial kernel matrix K :

$$I_{\text{filtered}}(x, y) = (I * K)(x, y) = \sum_{m=-k}^k \sum_{n=-k}^k I(x-m, y-n) K(m, n)$$

A common preprocessing smoothing filter is the discrete Gaussian kernel, which calculates a weighted local average across neighboring pixels, suppressing random sensor variations while preserving macroscopic image structure.

To detect structural boundaries, computer vision algorithms estimate directional image intensity gradients. The spatial gradient vector ∇I indicates the magnitude and direction of the sharpest local intensity change:

$$\nabla I = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix}$$

The gradient magnitude $G = \sqrt{G_x^2 + G_y^2}$ identifies candidate edge pixels. Standard differential operators, such as the Sobel operator, use small 3×3 derivative convolution masks to calculate numerical gradient approximations along horizontal and vertical axes.

7.1.2 Feature Extraction and Image Segmentation

Image segmentation is the computational process of partitioning a digital image into multiple non-overlapping spatial regions or sets of pixels based on shared perceptual properties. Segmentation separates foreground objects of interest from background noise.

Thresholding techniques partition an image by comparing individual pixel intensities to a selected cut-off value T :

$$g(x, y) = \begin{cases} 1 & \text{if } I(x, y) \geq T \\ 0 & \text{if } I(x, y) < T \end{cases}$$

Global thresholding algorithms, such as Otsu’s method, automatically select an optimal threshold value T by maximizing the variance between pixel classes. Advanced segmentation techniques use edge-based boundary detection, region-growing heuristics, or graph-cut optimization to group pixels that share consistent color, intensity, or texture properties.

Once regions are segmented, feature extraction algorithms convert spatial regions into quantitative descriptor vectors. Local scale-invariant feature descriptors, such as the Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG), calculate localized gradient orientation distributions. These feature vectors remain robust against linear illumination changes, scaling, rotation, and minor geometric distortions.

7.1.3 Object Recognition Pipelines

An object recognition pipeline converts raw sensory pixel inputs into discrete symbolic class labels and spatial bounding boxes. Figure 7.1 illustrates the sequential processing stages involved in a standard visual perception system.

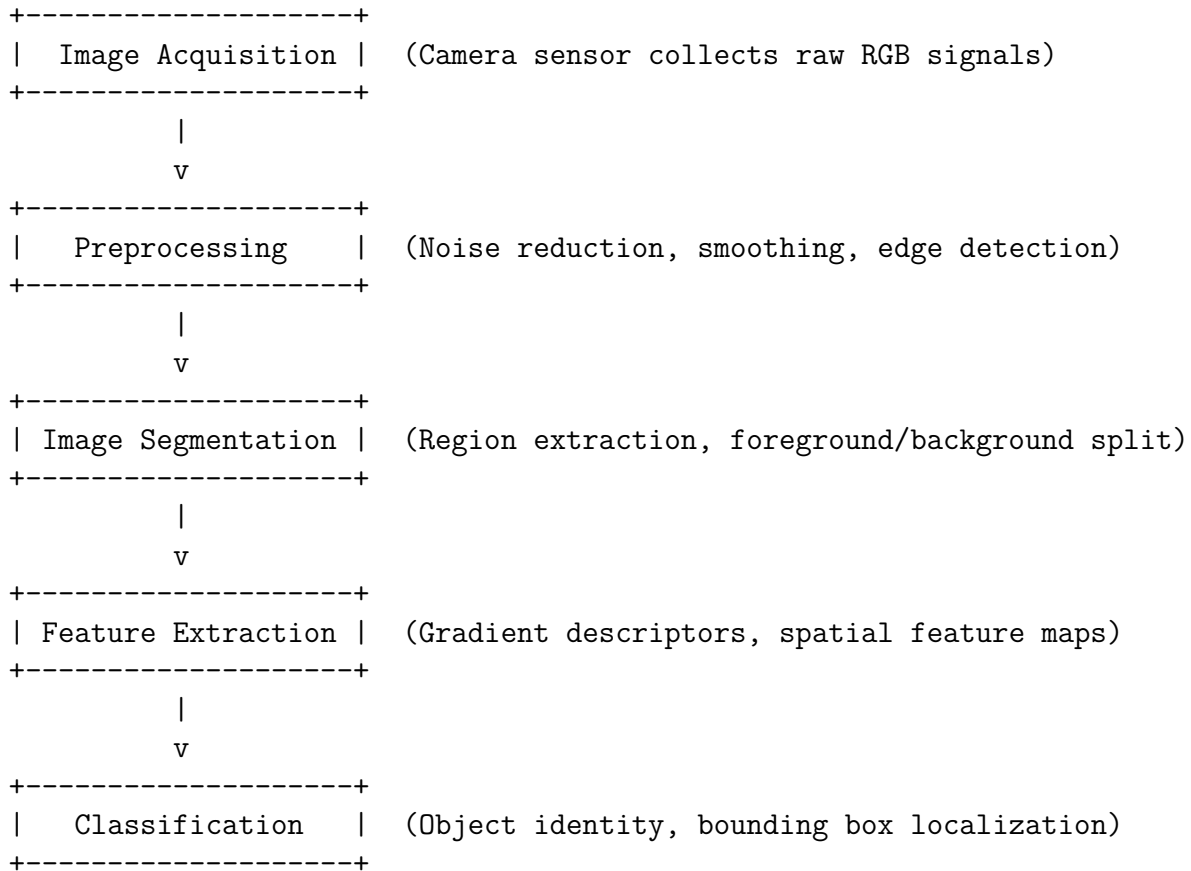


Figure 7.1: Sequential processing stages in an object recognition pipeline.

In modern implementations, classical manual feature extraction steps are often integrated directly into unified deep convolutional networks, where early linear layer filters extract primitive edges, middle layers group low-level edges into geometric textures, and deep layers capture complete object concepts.

7.2 Facial Recognition Algorithms

Facial recognition is a specialized subfield of computer vision that identifies or verifies human individuals by analyzing spatial patterns of facial features. The problem requires normalizing variable lighting conditions, pose angles, facial expressions, and partial occlusions.

7.2.1 Classical Statistical Approaches: Eigenfaces

Eigenfaces is a statistical method for facial recognition that constructs a low-dimensional representation of face images using Principal Component Analysis (PCA). Developed by Matthew Turk and Alex Pentland, this approach treats an entire face image as a single point in a high-dimensional vector space and projects it onto an optimal orthogonal subspace that preserves maximum variance across training samples.

Consider a training dataset of M face images, where each image is flattened into a single N -dimensional vector $\mathbf{x}_i \in \mathbb{R}^N$ (for an image of size $w \times h$, $N = w \times h$). The average or mean face vector $\boldsymbol{\mu}$ across all training samples is calculated as:

$$\boldsymbol{\mu} = \frac{1}{M} \sum_{i=1}^M \mathbf{x}_i$$

Each face vector is normalized by subtracting the mean face, producing a mean-subtracted vector $\boldsymbol{\phi}_i = \mathbf{x}_i - \boldsymbol{\mu}$. These normalized vectors are aggregated column-wise into an $N \times M$ matrix $A = [\boldsymbol{\phi}_1, \boldsymbol{\phi}_2, \dots, \boldsymbol{\phi}_M]$. The sample covariance matrix C is defined by:

$$C = \frac{1}{M} \sum_{i=1}^M \boldsymbol{\phi}_i \boldsymbol{\phi}_i^T = \frac{1}{M} A A^T$$

The covariance matrix C has dimensions $N \times N$. Computing the N eigenvectors directly for high-resolution images (where N can exceed 10^5) is computationally intractable. To resolve this, PCA computes the eigenvectors v_k of the smaller $M \times M$ matrix $L = A^T A$:

$$A^T A v_k = \lambda_k v_k$$

Multiplying both sides of this equation from the left by A yields:

$$A A^T (A v_k) = \lambda_k (A v_k)$$

This proves that $u_k = Av_k$ is an eigenvector of the original high-dimensional covariance matrix $C = \frac{1}{M}AA^T$, associated with the eigenvalue λ_k . The calculated orthogonal vectors u_k are the eigenfaces. Selecting the top K eigenvectors corresponding to the largest eigenvalues creates a reduced basis space. Any face image $\mathbf{x}$ can be approximated by projecting its mean-centered vector $\boldsymbol{\phi} = \mathbf{x} - \boldsymbol{\mu}$ onto this subspace, producing a compact weight vector $\mathbf{w} = [w_1, w_2, \dots, w_K]^T$, where $w_k = u_k^T \boldsymbol{\phi}$.

7.2.2 Feature-Based Detection: The Viola-Jones Framework

The Viola-Jones framework provides fast, accurate face detection in real-time video streams. The framework incorporates four computational innovations:

1. **Haar-like Features:** Simple rectangular digital filters that calculate scalar intensity differences between adjacent rectangular image regions.
2. **Integral Image Representation:** An intermediate spatial representation where the value at pixel location (x, y) contains the sum of all pixel intensities above and to the left of (x, y) . This allows the sum of intensities over any arbitrary rectangle to be computed in $O(1)$ constant time using four array lookups.
3. **AdaBoost Machine Learning:** A boosting algorithm that selects a tiny subset of critical Haar-like features out of tens of thousands of candidates, combining them into an effective weak classifier ensemble.
4. **Cascaded Attentional Structure:** A multi-stage cascade of increasingly complex classifiers. Early stages discard obvious non-face background regions instantaneously with minimal computational cost, reserving complex evaluations for promising candidate face windows.

Figure 7.2 illustrates the evaluation process of candidate sub-windows through the attentional classifier cascade.

7.2.3 Deep Learning Approaches: Convolutional Neural Networks and Face Embeddings

Modern facial recognition systems rely on deep Convolutional Neural Networks (CNNs). Rather than using linear spatial projections or handcrafted rectangular features, CNNs learn multi-layer non-linear abstractions directly from raw pixel matrices.

Deep architectures (such as ResNet or MobileNet) map face images into a continuous high-dimensional vector space (typically $\mathbb{R}^{128}$ or $\mathbb{R}^{512}$), known as a face embedding. The network is trained using metric learning loss functions, such as triplet loss:

$$\mathcal{L} = \max \left(0, \|\mathbf{f}(A) - \mathbf{f}(P)\|_2^2 - \|\mathbf{f}(A) - \mathbf{f}(N)\|_2^2 + \alpha \right)$$

In this formulation, $\mathbf{f}(A)$ represents the embedding vector of an anchor face, $\mathbf{f}(P)$ represents a positive face sample (same person), $\mathbf{f}(N)$ represents a negative face sample (different person), and α is a predefined enforcement margin. The loss function minimizes the squared

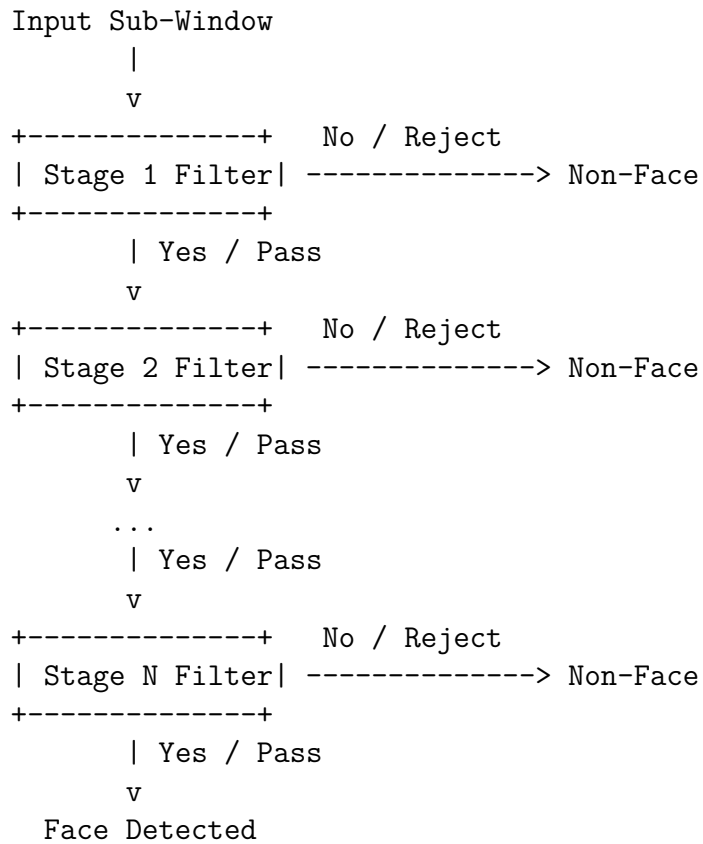


Figure 7.2: Cascaded attentional classifier structure in the Viola-Jones framework.

Euclidean distance between face embeddings of the same individual while maximizing the distance between embeddings of different individuals. Once trained, face recognition reduces to calculating vector distances between face embeddings using standard distance metrics.

Feature / Dimension	Eigenfaces (PCA)	Viola-Jones Framework	Deep CNN Embeddings
Primary Approach	Global linear subspace transformation	Cascaded boosting of simple Haar-like features	Deep non-linear hierarchical feature learning
Representation Level	Low-dimensional global pixel combinations	Local spatial edge and bar features	Multi-scale abstract semantic representations
Inference Speed	Extremely fast after dimensionality reduction	Real-time CPU performance (> 30 FPS)	Fast execution, optimal on GPU hardware acceleration
Illumination Sensitivity	High (sensitive to lighting changes)	Moderate (handles uniform lighting adjustments)	Low (robust against lighting, angle, and occlusion)
Primary Application	Small-scale baseline face recognition	Real-time face window detection	Biometric identification and verification systems

Table 7.1: Comparison of major facial recognition paradigms.

7.2.4 Worked Example: Eigenface Computation and Projection

To demonstrate the mathematical steps of the Eigenfaces approach, consider a simplified visual system processing 2×2 pixel grayscale images ($N = 4$ pixels).

Step 1: Define Training Dataset

Assume a training set consisting of $M = 3$ normalized face image vectors:

$$\mathbf{x}_1 = \begin{bmatrix} 2 \\ 4 \\ 2 \\ 4 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 4 \\ 2 \\ 4 \\ 2 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} 3 \\ 3 \\ 3 \\ 3 \end{bmatrix}$$

Step 2: Calculate Mean Face Vector

Compute the mean face vector $\boldsymbol{\mu}$:

$$\boldsymbol{\mu} = \frac{1}{3}(\mathbf{x}_1 + \mathbf{x}_2 + \mathbf{x}_3) = \frac{1}{3} \begin{bmatrix} 2 + 4 + 3 \\ 4 + 2 + 3 \\ 2 + 4 + 3 \\ 4 + 2 + 3 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 3 \\ 3 \end{bmatrix}$$

Step 3: Construct Mean-Centered Matrix

Subtract $\boldsymbol{\mu}$ from each image vector to determine $\boldsymbol{\phi}_i$:

$$\boldsymbol{\phi}_1 = \mathbf{x}_1 - \boldsymbol{\mu} = \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix}, \quad \boldsymbol{\phi}_2 = \mathbf{x}_2 - \boldsymbol{\mu} = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}, \quad \boldsymbol{\phi}_3 = \mathbf{x}_3 - \boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Assemble matrix A :

$$A = \begin{bmatrix} -1 & 1 & 0 \\ 1 & -1 & 0 \\ -1 & 1 & 0 \\ 1 & -1 & 0 \end{bmatrix}$$

Step 4: Compute Eigenvalues and Eigenvectors of Matrix $L = A^T A$

Calculate the 3×3 inner product matrix L :

$$L = A^T A = \begin{bmatrix} -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 1 & 0 \\ 1 & -1 & 0 \\ -1 & 1 & 0 \\ 1 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 4 & -4 & 0 \\ -4 & 4 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

To find the eigenvalues of L , solve $\det(L - \lambda I) = 0$:

$$\det \begin{bmatrix} 4 - \lambda & -4 & 0 \\ -4 & 4 - \lambda & 0 \\ 0 & 0 & -\lambda \end{bmatrix} = (-\lambda) [(4 - \lambda)^2 - 16] = -\lambda(\lambda^2 - 8\lambda) = -\lambda^2(\lambda - 8) = 0$$

The eigenvalues are $\lambda_1 = 8, \lambda_2 = 0, \lambda_3 = 0$.

For the primary eigenvalue $\lambda_1 = 8$, solve $(L - 8I)\mathbf{v}_1 = \mathbf{0}$:

$$\begin{bmatrix} -4 & -4 & 0 \\ -4 & -4 & 0 \\ 0 & 0 & -8 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \\ v_{13} \end{bmatrix} = \mathbf{0} \implies v_{13} = 0, \quad v_{11} = -v_{12}$$

Choosing a unit-length eigenvector yields $\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}$.

Step 5: Compute Eigenfaces and Normalize

Map $\mathbf{v}_1$ back to the original image space using $\mathbf{u}'_1 = A\mathbf{v}_1$:

$$\mathbf{u}'_1 = \begin{bmatrix} -1 & 1 & 0 \\ 1 & -1 & 0 \\ -1 & 1 & 0 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \end{bmatrix} = \begin{bmatrix} -2/\sqrt{2} \\ 2/\sqrt{2} \\ -2/\sqrt{2} \\ 2/\sqrt{2} \end{bmatrix} = \sqrt{2} \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix}$$

Calculate the Euclidean length of $\mathbf{u}'_1$: $\|\mathbf{u}'_1\| = \sqrt{2 \cdot (1 + 1 + 1 + 1)} = \sqrt{8} = 2\sqrt{2}$.

Normalizing $\mathbf{u}'_1$ gives the primary eigenface $\hat{\mathbf{u}}_1$:

$$\hat{\mathbf{u}}_1 = \frac{\mathbf{u}'_1}{\|\mathbf{u}'_1\|} = \frac{1}{2} \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix}$$

Step 6: Face Projection and Reconstruction

Consider a new test image vector $\mathbf{x}_{\text{test}} = \begin{bmatrix} 1 \\ 5 \\ 1 \\ 5 \end{bmatrix}$. First, center the test image:

$$\boldsymbol{\phi}_{\text{test}} = \mathbf{x}_{\text{test}} - \boldsymbol{\mu} = \begin{bmatrix} 1 - 3 \\ 5 - 3 \\ 1 - 3 \\ 5 - 3 \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \\ -2 \\ 2 \end{bmatrix}$$

Project $\boldsymbol{\phi}_{\text{test}}$ onto the eigenface subspace to obtain weight w_1 :

$$w_1 = \hat{\mathbf{u}}_1^T \boldsymbol{\phi}_{\text{test}} = \frac{1}{2} \begin{bmatrix} -1 & 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} -2 \\ 2 \\ -2 \\ 2 \end{bmatrix} = \frac{1}{2} (2 + 2 + 2 + 2) = 4$$

Reconstruct the estimated face image $\hat{\mathbf{x}}$ using the weight coordinate:

$$\hat{\mathbf{x}} = \boldsymbol{\mu} + w_1 \hat{\mathbf{u}}_1 = \begin{bmatrix} 3 \\ 3 \\ 3 \\ 3 \end{bmatrix} + 4 \cdot \frac{1}{2} \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 - 2 \\ 3 + 2 \\ 3 - 2 \\ 3 + 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \\ 1 \\ 5 \end{bmatrix}$$

The single weight scalar $w_1 = 4$ accurately captures the primary variance structure of the input image in the low-dimensional eigenface space.

7.3 Robot Programming and Control

Robotics is an interdisciplinary branch of engineering and computer science focused on designing, constructing, operating, and programming physical agents that interact directly with the physical world. While computer vision acts as an visual perception system, robotics governs mechanical actuation, movement control, and real-time environment interaction.

7.3.1 Fundamental Concepts in Robotics and Physical Sensing

A robot operates through a continuous control cycle: it measures environmental state variables via sensors, processes these signals within a computational control architecture, and commands physical movements via actuators.

Sensors fall into two general functional categories:

1. **Proprioceptive Sensors:** Measure internal states of the robot system. Examples include optical encoders measuring motor shaft angles, potentiometers, and Inertial Measurement Units (IMUs) measuring linear acceleration and angular velocity.
2. **Exteroceptive Sensors:** Observe external environmental conditions surrounding the robot. Examples include LiDAR rangefinders, ultrasonic distance sensors, stereo vision cameras, and tactile force sensors.

Actuators transform electrical or pneumatic energy into physical forces and movement. Electric DC motors, stepper motors, and servo actuators drive robotic joints, while end-effectors (such as mechanical grippers, welding torches, or suction cups) allow a robot arm to manipulate objects directly.

7.3.2 Forward and Inverse Kinematics

Kinematics is the branch of mechanics and robotics that mathematically models the geometry of motion of link mechanisms without considering the forces that cause the motion.

A manipulator arm consists of a chain of rigid structural links connected by flexible joints (revolute rotary joints or prismatic sliding joints). The joint configuration is defined by a vector of joint variables $\boldsymbol{\theta} = [\theta_1, \theta_2, \dots, \theta_n]^T$. The position and orientation of the end-effector in 3D Cartesian workspace coordinates is represented as a spatial pose vector $\mathbf{x} = [x, y, z, \alpha, \beta, \gamma]^T$.

Forward Kinematics (FK) calculates the spatial pose of the end-effector $\mathbf{x}$ given a vector of specified joint parameters $\boldsymbol{\theta}$:

$$\mathbf{x} = f(\boldsymbol{\theta})$$

Forward kinematics yields a unique solution for any given set of joint angles, easily computed by multiplying coordinate transformation matrices using Denavit-Hartenberg (D-H) matrix conventions.

Inverse Kinematics (IK) solves the reverse problem: determining the necessary vector of joint parameters $\boldsymbol{\theta}$ required to place the end-effector at a desired target Cartesian pose $\mathbf{x}$:

$$\boldsymbol{\theta} = f^{-1}(\mathbf{x})$$

Inverse kinematics is computationally more difficult than forward kinematics. The inverse function f^{-1} is non-linear and may yield multiple valid joint configurations for a single target position (such as elbow-up versus elbow-down configurations), or no mathematical solution if the target point lies outside the arm's physical workspace.

To illustrate, consider a two-link planar arm operating in a 2D workspace, where link 1 has length L_1 and joint angle θ_1 , and link 2 has length L_2 and relative joint angle θ_2 . Figure 7.3 illustrates this geometric structure.

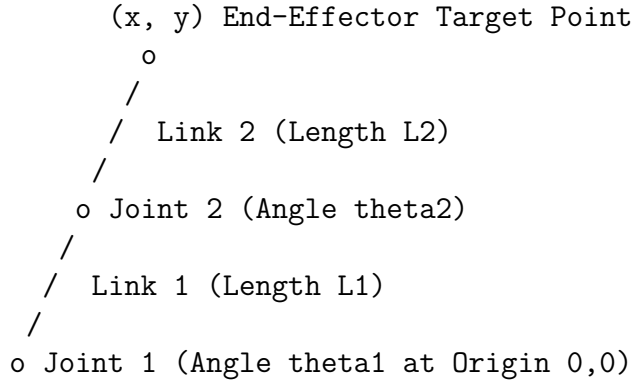


Figure 7.3: Geometric configuration of a two-link planar manipulator arm.

The Forward Kinematics algebraic equations for the end-effector position (x, y) are derived directly from spatial trigonometry:

$$\begin{aligned} x &= L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \\ y &= L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

To solve the Inverse Kinematics problem for specified end-effector coordinates (x, y) , we calculate the squared distance $R^2 = x^2 + y^2$ from the origin:

$$x^2 + y^2 = (L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2))^2 + (L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2))^2 = L_1^2 + L_2^2 + 2L_1L_2 \cos(\theta_2)$$

Solving for $\cos(\theta_2)$ yields:

$$\cos(\theta_2) = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

Using trigonometric identities, θ_2 can be calculated as:

$$\theta_2 = \pm \arccos \left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2} \right)$$

The positive and negative sign options represent the elbow-down and elbow-up geometric solutions. Once θ_2 is determined, joint angle θ_1 is obtained via:

$$\theta_1 = \arctan 2(y, x) - \arctan 2(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2)$$

7.3.3 Robot Programming Paradigms and Architecture

Robotics architectures structure how sensory inputs are coupled to motor actions. Three major control paradigms exist:

1. **Sense-Plan-Act (Deliberative Architecture):** A sequential paradigm where the system builds a detailed world model from sensor data, plans a complete trajectory using search algorithms, and executes the plan. This approach is well suited for static environments but responds slowly to sudden dynamic changes.
2. **Reactive Architecture (Subsumption):** Proposed by Rodney Brooks, this approach eliminates explicit global world models and long-term planning modules. Simple sensory-motor feedback loops operate concurrently. High-level behaviors suppress lower-level behaviors directly, ensuring fast real-time responses to environment changes.
3. **Hybrid Architecture:** Combines reactive control loops for immediate obstacle avoidance with deliberative planning modules for long-term goal trajectory calculation.

Modern robotics systems heavily use message-passing software frameworks such as the Robot Operating System (ROS). ROS coordinates execution across distributed computational processes called nodes, which communicate asynchronously over named channels termed topics using a publish-subscribe pattern. Figure 7.4 contrasts the classic Sense-Plan-Act control loop with a distributed ROS event-driven architecture.

7.4 Practical Robotics Applications

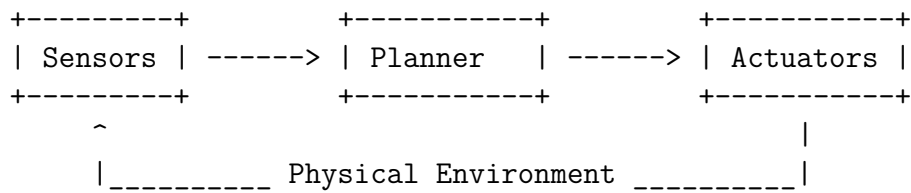
The synthesis of computer vision and programmable robotics enables autonomous agents to operate reliably across industrial, commercial, and unstructured physical domains.

7.4.1 Industrial and Manufacturing Automation

Industrial robotics uses articulated arm manipulators to perform repetitive high-precision tasks. Traditional industrial robots followed rigid pre-programmed joint movement sequences. Integrating real-time vision pipelines enables flexible vision-guided robotics:

- **Automated Quality Control Inspection:** High-speed industrial camera arrays capture manufactured parts on assembly lines, applying edge extraction and template matching algorithms to detect surface defects and micro-cracks under 100 milliseconds.
- **Bin Picking and Sorting:** Vision systems locate unstructured items randomly piled in containers, estimate 3D spatial poses, and feed target coordinate matrices to inverse kinematics motion planners to guide pick-and-place grippers.

Sense-Plan-Act Deliberative Loop:



ROS Modular Node Architecture:

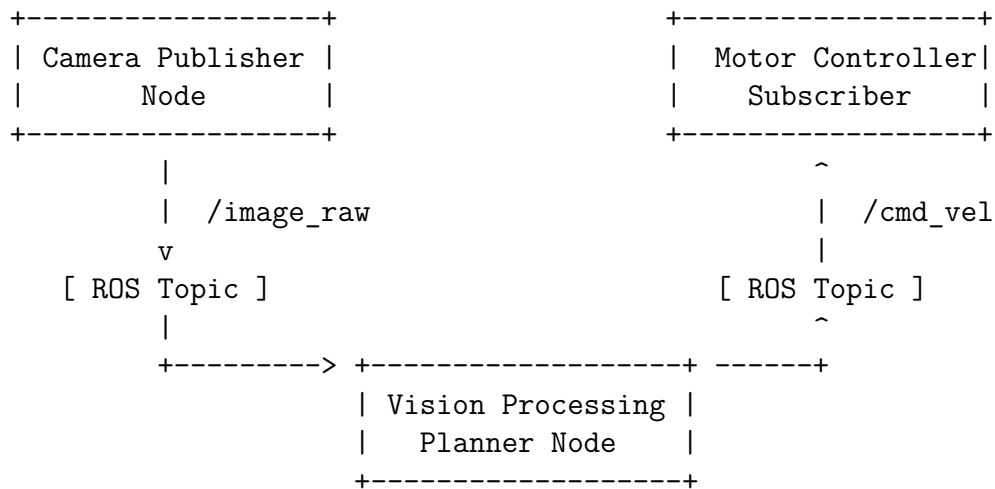


Figure 7.4: Comparison of Sense-Plan-Act control loop and ROS node-based architecture.

7.4.2 Autonomous Mobile Robots and Perception Systems

Autonomous Mobile Robots (AMRs) navigate through unstructured physical environments without human intervention. AMRs rely on Simultaneous Localization and Mapping (SLAM) algorithms. SLAM constructs a geometric spatial map of an unknown environment while tracking the robot’s real-time position within that growing map.

In automated logistics, vision-guided warehouse robots navigate material facilities, using LiDAR range sensors and depth-sensing cameras to avoid dynamic obstacles like human workers. Autonomous Ground Vehicles (AGVs) use deep neural network vision models to detect lane markers, interpret traffic signals, and identify pedestrians across variable weather conditions.

7.4.3 Ethics, Challenges, and Future Trends

Deploying vision-guided robotic systems introduces technical and societal challenges:

- **Algorithmic Bias and Fairness:** Facial recognition systems trained on imbalanced datasets show elevated error rates across demographic sub-groups, raising critical civil rights concerns in public surveillance and law enforcement contexts.
- **Physical Safety in Human-Robot Interaction:** Collaborative robots (cobots) operate alongside human operators without traditional physical safety barriers. Ensuring physical safety requires low-latency computer vision tracking and force-limiting control systems to prevent collisions.
- **Privacy Concerns:** Continuous video recording and real-time biometric identification in urban public spaces create massive surveillance capabilities, requiring clear regulatory frameworks and privacy protections.

7.5 Summary and Bridge to Advanced AI Topics

This chapter explored the perceptual and physical dimensions of artificial intelligence, examining how computer vision transforms raw pixel matrices into structured scene representations and how robotics converts target workspace goals into physical joint movements. Classical methods such as Eigenfaces and analytical inverse kinematics provide mathematical foundations, while deep convolutional neural networks and distributed software architectures support dynamic real-time operations.

With this chapter, the core computational components of artificial intelligence—perception, search, knowledge representation, reasoning, natural language understanding, and physical actuation—are established. Future studies build upon these fundamentals to explore advanced topics, including multi-agent reinforcement learning, autonomous field robotics, and deep generative foundation models.

Tutorial Questions

1. Define **computer vision** and contrast its core objectives with traditional digital image processing. Provide one practical application where image processing alone is insufficient and full computer vision capabilities are required.
2. Explain the fundamental mathematical steps involved in the Eigenfaces face recognition algorithm. Why is Principal Component Analysis (PCA) applied to compute eigenvectors from the matrix $L = A^T A$ rather than computing the eigenvectors of the full sample covariance matrix C directly?
3. Given a simplified 2×2 pixel face image dataset with mean face vector $\boldsymbol{\mu} = [4, 4, 4, 4]^T$ and a normalized principal eigenface $\hat{\mathbf{u}}_1 = [0.5, -0.5, 0.5, -0.5]^T$:
 - (a) Compute the projection weight w_1 for an input test face vector $\mathbf{x}_{\text{test}} = [6, 2, 6, 2]^T$.
 - (b) Calculate the reconstructed image vector $\hat{\mathbf{x}}$ derived from this projection weight.
4. Compare and contrast the classical Viola-Jones facial detection architecture with modern deep Convolutional Neural Network (CNN) face embedding approaches. Highlight differences regarding feature engineering, inference speed, and robustness against illumination variations.
5. Distinguish clearly between **Forward Kinematics** and **Inverse Kinematics** in robotic manipulators. Why is solving the Inverse Kinematics problem generally more challenging than solving the Forward Kinematics problem?
6. A two-link planar robotic manipulator arm has link lengths $L_1 = 4$ units and $L_2 = 3$ units. Calculate the joint angle parameters (θ_1, θ_2) required to position the end-effector at Cartesian coordinates $(x = 0, y = 7)$. Show all algebraic steps.
7. Describe the structural differences between the traditional Sense-Plan-Act control paradigm and Rodney Brooks' Reactive Subsumption architecture. Explain how modern robotic frameworks like ROS bridge these paradigms using node-based message passing.
8. Autonomous mobile vehicles rely heavily on computer vision and real-time range sensors to execute Simultaneous Localization and Mapping (SLAM). Discuss two technical challenges faced by vision-guided mobile robots operating in dynamic urban environments and propose appropriate computational solutions.